User Manual

TDS Toolkit

File, Waveform, Mask and Limit Test Software for TDS 500, TDS 600 and TDS 700 Series Digitizing Oscilloscopes

Version 1.0.0
Copyright © 2026 Jared Cabot. MIT Licence.

TDSTK-100

Copyright © 2026 Jared Cabot. All rights reserved.

This software is released under the MIT Licence. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files to deal in the software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and sell copies of the software, subject to the inclusion of the copyright notice and this permission notice in all copies or substantial portions of the software. The full text accompanies the program as LICENSE.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

TEKTRONIX, TEK and TDS are registered trademarks of Tektronix, Inc. Windows is a registered trademark of Microsoft Corporation. This program is not a Tektronix product, and neither the program nor this manual is endorsed by or affiliated with Tektronix, Inc.

Figures 1–1, 1–2, 1–3 and 8–3 are reproduced from Tektronix manuals 070-9876-00 and 071-0627-02 for the purpose of identifying the connectors and controls the reader is asked to use.

Table of Contents

Related Manuals xi

1 Getting Started 1–1

Product Description 1–1

What the Program Does 1–1

What Your Instrument Can Do 1–1

Requirements 1–2

Equipment Required 1–2

VISA Runtime 1–2

Connecting the Instrument 1–3

The GPIB Connector 1–3

Setting the Address 1–4

Installing the Program 1–4

From the Executable 1–4

From Source 1–5

Starting the Program 1–5

The First Connection 1–5

The Window 1–5

While the Program Is Busy 1–6

Conventions 1–7

Command Line 1–7

2 Files 2–1

The Panes 2–1

Folder Tree 2–1

File List 2–1

What Is Not Listed 2–2

Moving About 2–3

How Long a Listing Takes 2–3

Downloading Files 2–3

Uploading Files 2–4

Names on the Instrument 2–4

How Long an Upload Takes 2–5
Creating Folders 2–5
Deleting Files and Folders 2–5
The Instrument's Own Files 2–5
Formatting a Volume 2–6
Instruments With No Filesystem 2–6
What the Program Sends 2–7
3 Screenshot 3–1
Taking a Screenshot 3–2
Formats 3–2
Layout and Palette 3–3
Layout 3–3
Palette 3–3
Saving the Picture 3–4
What the Program Sends 3–4
4 Waveforms 4–1
The Source Lists 4–2
Capturing 4–2
Reading the Plot 4–3
Zoom and Pan 4–3
Saving a Waveform 4–5
Loading a Waveform 4–6
Sending a Waveform to the Instrument 4–6
Deleting a Reference 4–7
Protocol decode 4–8
Adding a protocol 4–8
What the Program Sends 4–9
5 Limits 5–1
Running a Limit Test 5–1
Learning From a Loaded Trace 5–2
The Toolbar 5–2
The Reference List 5–3
The Buttons Beside the Reference List 5–3
The Template Library 5–3
Tolerances 5–3
The Verdict 5–3

Drawing the Envelope 5–4
What a New Template Leaves on the Canvas 5–4
What the Program Sends 5–5
Learn Template 5–5
Start Test 5–5
Reading the Template Back 5–5
Sending a Drawn Envelope 5–6
6 Masks 6–1
The Toolbar 6–2
The Library 6–2
Drawing 6–3
The Tools 6–3
Keys 6–3
Typing Coordinates 6–4
Options Under the Canvas 6–4
What the Instrument Will Take 6–5
Sending a Mask 6–6
Judging a Signal 6–6
Setting Up an Eye 6–6
With a Clock 6–7
Without a Clock 6–8
Checking the Sweep 6–8
Mask Files 6–8
What the Program Sends 6–9
Judging a Signal 6–9
7 Error Log 7–1
Reading the Log 7–1
Memory Faults 7–2
Saving the Log 7–2
Clearing the Log 7–2
Instruments With No Error Log 7–3
What the Program Sends 7–3
8 System 8–1
Identity 8–2
Date and Time 8–2
Hardcopy and Ports 8–2

Calibration and Self Test 8–3
Signal Path Compensation 8–3
Self Test 8–3
Reading a Failure 8–3
Memory 8–5
Factory Options 8–6
The Procedure 8–6
The Option Words 8–7
What the Program Sends 8–9
Reading the Tab 8–9
Identity 8–9
Clock and Ports 8–9
Calibration and Self Test 8–9
Memory 8–9
Factory Options 8–9
9 Backup 9–1
What Goes In 9–2
The Backup File 9–2
Verifying 9–3
Restoring 9–3
Calibration Constants 9–4
Backing Up 9–4
The Checksum 9–4
Restoring 9–4
Written Is Not Stored 9–5
The NVRAM 9–6
Backing Up 9–6
What the Checksums Say 9–6
Restoring 9–7
Why Less Is Written Than the File Holds 9–7
What the Program Sends 9–8
Calibration Constants 9–8
The NVRAM 9–8
10 Firmware 10–1
Preparing the Instrument 10–2
The Protection Switch 10–2

Starting the Monitor 10–2
Controls 10–2
Firmware Images 10–3
Where They Are Kept 10–3
Which Version to Write 10–3
One Image, Several Models 10–3
Backups 10–4
Writing 10–4
The Sequence 10–4
Fast and Slow Writes 10–4
Recovery 10–5
11 Settings 11–1
Plot Colours 11–1
Presets 11–2
Decode colours 11–2
Saved Pictures 11–3
Editing 11–3
Language 11–3
Where Things Are Kept 11–4
A Firmware Capability i
Files and Transfers i
Factory Options v
B File Formats i
Waveforms i
What a .WFM Carries ii
Masks and Setups ii
Screenshots iii
The Program's Own Files iii
C If Something Goes Wrong i
Connecting i
Transfers i
Waveforms and Screens ii
Masks and Limits ii
The Log iii

List of Figures

Figure 1–1: GPIB connector location on the rear panel 1–3
Figure 1–2: Stacking GPIB connectors 1–3
Figure 1–3: Typical GPIB network configurations 1–4
Figure 1–4: The program window, showing the Files tab 1–6
Figure 2–1: The Files tab 2–1
Figure 3–1: The Screenshot tab 3–1
Figure 4–1: The Waveforms tab, decoding a captured RS-232 signal 4–1
Figure 5–1: The Limits tab, showing a template learnt from a square wave 5–1
Figure 6–1: The Masks tab, with a USB full speed mask open 6–1
Figure 6–2: The Start measurement dialog for an eye mask 6–7
Figure 7–1: The Error Log tab 7–1
Figure 8–1: The System tab 8–1
Figure 8–2: The Factory options dialog 8–6
Figure 8–3: Accessing the protection switch, from Tektronix service manual 071-0627-02 8–7
Figure 9–1: The Backup tab 9–1
Figure 10–1: The Firmware tab 10–1
Figure 11–1: The Settings tab 11–1

List of Tables

Table 1–1: What each firmware generation offers 1–2
Table 1–2: Equipment required 1–2
Table 1–3: Command line switches 1–7
Table 2–1: Navigation keys 2–3
Table 2–2: Files the program asks about twice 2–6
Table 3–1: Image formats 3–2
Table 3–2: Measured capture size and time 3–3
Table 3–3: Palette choices 3–3
Table 4–1: Mouse and keyboard on the plot 4–4
Table 4–2: What each format keeps 4–5
Table 5–1: What the verdict means 5–4
Table 6–1: What the library lists 6–2
Table 6–2: The tool palette 6–3
Table 6–3: Keys in the editor 6–4
Table 6–4: Mask limits 6–5
Table 8–1: Hardcopy and RS-232 settings 8–2
Table 8–2: The factory option words 8–8
Table 10–1: Complete operation, measured 10–5
Table 11–1: What each colour draws 11–2
Table A–1: Filesystem and transfer commands by firmware ii
Table A–2: Filesystem and transfer commands by firmware (cont.) iii
Table A–3: Filesystem and transfer commands by firmware (cont.) iv
Table A–4: Option codes each firmware reads v
Table A–5: Option codes each firmware reads (cont.) vi
Table A–6: Option codes each firmware reads (cont.) vii
Table B–1: Waveform formats i
Table B–2: The fields in a .WFM ii
Table B–3: Mask and setup formats ii
Table B–4: Mask limits iii
Table B–5: Files beside the program iii

Table C–1: The instrument does not answer i
Table C–2: Files and transfers i
Table C–3: Capture ii
Table C–4: Testing ii

General Safety Summary

Review the following safety precautions to avoid injury and prevent damage to the oscilloscope or to any product connected to it. To avoid potential hazards, use the oscilloscope only as specified in its own manuals, and use this program only as specified here.

Only qualified personnel should perform service procedures. Reaching the memory protection switch described in Section 8 requires access to the oscilloscope's cabinet.

To Avoid Fire or Personal Injury

Ground the Product. The oscilloscope is grounded through the grounding conductor of its power cord, and the GPIB cable screen bonds its chassis to the controller. To avoid electric shock, both instruments must be connected to earth ground before any signal connection is made.

Connect and Disconnect Properly. Where the two instruments are powered from different supplies, do not connect or disconnect the GPIB cable while either one is switched on. Connect the controller's cable to the oscilloscope before applying power to the signal source.

Do Not Operate Without Covers. Do not operate the oscilloscope with covers or panels removed. The memory protection switch is reached from outside the cabinet; nothing in this manual requires the cabinet to be opened with power applied.

Observe All Terminal Ratings. This program can switch channels on and change vertical scale factors over the bus. It cannot know what is connected to the inputs. Observe the ratings marked on the oscilloscope.

Do Not Operate With Suspected Failures. If the oscilloscope reports diagnostic failures on the System tab, have it inspected by qualified service personnel before relying on any measurement it makes.

Data Loss Summary

This program writes to the oscilloscope. The following actions change or destroy data on the instrument and cannot be undone from this program. Each is described in full where it appears.

Irreversible Actions

Delete. Deleting a file or a folder on the instrument is final. There is no recycle bin on the instrument, and removing a folder removes everything inside it. See *Deleting Files and Folders* in Section 2.

Format. Formatting a volume empties it. On the hard disk this includes the Java runtime and every application shipped with the instrument, and none of them will run until they are loaded back over GPIB. See *Formatting a Volume* in Section 2.

Upload. Uploading a file replaces any file of the same name without further warning, because the program sets FILESYSTEM:OVERWRITE ON when it connects. See *Uploading Files* in Section 2.

Send to Instrument. Writing a waveform into a reference memory replaces whatever that reference held, and writing one to the instrument's disk replaces any file of the same name. See *Sending a Waveform to the Instrument* in Section 4.

Learn Template, and the down arrow beside the reference list. Both write a limit template into a reference memory, replacing its contents. See Section 5.

Clear Errors. Clearing the instrument's error log erases its error history, which is kept in non-volatile memory. See Section 7.

Secure Erase. A secure erase writes zeroes over every waveform and setup memory. Calibration data is left untouched. See Section 8.

Recall Factory Setup. This returns the instrument to its default settings. The GPIB address, the calibration constants and protected user data are left untouched. See Section 8.

Options. Writing the factory option words writes directly to non-volatile memory. Nothing can be read back to confirm what was written. See Section 8.

Restore. Restoring a backup replaces the instrument's settings and overwrites any file on its disk that has the same name as one in the backup. There is no undo. See Section 9.

Preface

This is the User Manual for TDS Toolkit, a program that operates a Tektronix TDS 500, TDS 600 or TDS 700 series digitizing oscilloscope from a computer over the General Purpose Interface Bus. It describes what each of the program's ten tabs does, what each control sends to the instrument, and what the instrument does in reply.

The program is written for the instruments that carry the filesystem, hardcopy, mask and limit-test subsystems described in the TDS Family Programmer Manual. Which of those an individual instrument has depends on its firmware. The program establishes that when it connects and offers only what the instrument can do; Appendix A lists what was found in each firmware image.

The firmware of the following models was read directly, and what each version can do is listed in Appendix A. Other instruments in the same families also work; the program queries them when they connect instead of looking them up.

TDS 500: TDS 520, TDS 520A, TDS 520B, TDS 520C, TDS 524A, TDS 540, TDS 540A, TDS 540B, TDS 540C, TDS 544A, TDS 580C

TDS 600: TDS 620, TDS 620A, TDS 620B, TDS 640A, TDS 644A, TDS 644B, TDS 680B, TDS 684B, TDS 684C

TDS 700: TDS 714L, TDS 744A, TDS 754C, TDS 754D, TDS 784C, TDS 784D, TDS 794D

This manual assumes you can operate the oscilloscope from its front panel and that its GPIB interface is configured. It does not repeat the instrument's own documentation.

Related Manuals

The following Tektronix documents describe the instrument itself and the commands this program sends:

■ *TDS Family Digitizing Oscilloscopes Programmer Manual*, 070-9876-00, describes every command quoted in this manual.

■ *TDS 500D, TDS 600C, TDS 700D & TDS 714L User Manual*, 071-0130-03, describes operating the oscilloscope from its front panel.

■ *TDS 500D, TDS 600C, TDS 700D & TDS 714L Service Manual*, 071-0627-02, gives the location of the memory protection switch used in Section 8.

■ *TDS 500D, TDS 600C, TDS 700D & TDS 714L Performance Verification and Specifications*, 071-0630-03, gives the specifications the instrument is warranted to.

Getting Started

Product Description

What the Program Does

TDS Toolkit operates a TDS 500, TDS 600 or TDS 700 series digitizing oscilloscope from a computer over GPIB. The instruments carry a filesystem, a hardcopy subsystem, a mask subsystem and a limit-test subsystem. All of them can be reached over the bus, but the instrument provides no way of working with them from a computer.

The program is arranged as ten tabs. Each is described in its own section of this manual:

- **Files** browses the instrument's drives and moves files either way. See Section 2.
- **Screenshot** captures the instrument's display through its hardcopy port. See Section 3.
- **Waveforms** captures live channels and stored references, plots them, and writes them back. See Section 4.
- **Limits** builds a limit template and runs a limit test against it. See Section 5.
- **Masks** draws, sends and judges masks, and sets the instrument up for an eye diagram. See Section 6.
- **Error Log** reads the instrument's own error history. See Section 7.
- **System** reads and sets the instrument's identity, clock, ports, diagnostics and factory options. See Section 8.
- **Backup** copies the instrument's disk, settings, references and calibration constants into one file, and writes them back. See Section 9.
- **Firmware** writes a firmware image through the instrument's ROM monitor. See Section 10.
- **Settings** holds the plot colours and the saved-picture resolution. See Section 11.

Every command the program sends is described in the instrument's own Programmer Manual, with one exception: the factory option words in Section 8 appear in no Tektronix manual and were worked out on the bench.

What Your Instrument Can Do

What is possible depends on the firmware the instrument is running. The range covers nine years and three generations, and they do not behave alike:

Table 1–1: What each firmware generation offers

Firmware	Browse	Download	Upload
v2.x (TDS 520, 540, 620)	no filesystem	no	no
A and B series, v3.x to v4.x	yes	yes	no
C and D series, v5.0e and later	yes	yes	yes

The A and B series have no FILESYSTEM:WRITEFILE command, so no program can send a file to them over GPIB. They browse and download normally. Waveform transfer works on every instrument in the range, including those with no filesystem at all.

NOTE. *When it connects, the program establishes what the instrument can do and greys out anything its firmware cannot do, so an operation that could not complete is never offered. Appendix A gives the full table.*

Requirements

Table 1–2: Equipment required

Item	Minimum requirement
Oscilloscope	TDS 500, TDS 600 or TDS 700 series with the GPIB interface
GPIB controller	Any controller with a VISA driver, for example a National Instruments GPIB-USB-HS or a Keysight 82357B
GPIB cable	IEEE Std 488.1-1987, Tektronix part number 012-0991-00
Computer	Windows, with a VISA runtime installed

VISA Runtime

A VISA runtime is a system driver, not a part of this program, and nothing can reach the instrument without one. Install either NI-VISA or the Keysight IO Libraries Suite, and install the GPIB controller's own driver first, because VISA discovers the hardware through it.



CAUTION. *Install one VISA runtime, not two. Two implementations on one computer compete for the same interface and the symptom is an instrument that answers intermittently or not at all.*

Connecting the Instrument

The GPIB Connector

The oscilloscope has a 24-pin GPIB connector on its rear panel. The connector has a D-type shell and conforms to IEEE Std 488.1-1987. See Figure 1–1.

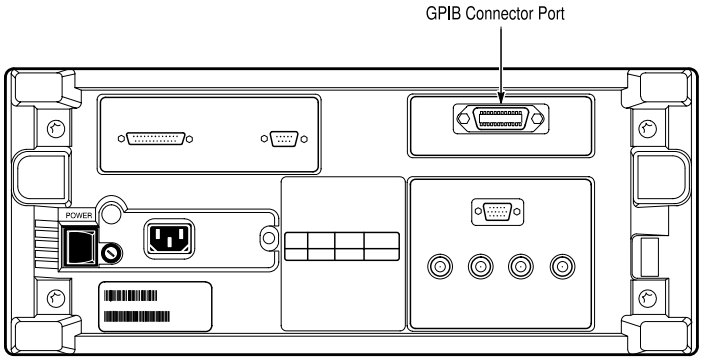


Figure 1–1: GPIB connector location on the rear panel

Attach the GPIB cable to this connector. Where more than one instrument is on the bus, the connectors stack: each cable's plug carries a socket on its back so the next cable can be attached to it. See Figure 1–2.

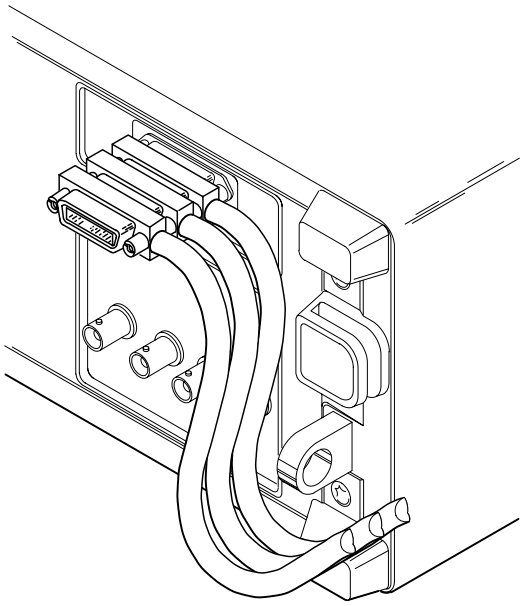


Figure 1–2: Stacking GPIB connectors

Observe the bus rules given in the instrument's own manual: no more than 15 devices on one bus, no more than 20 metres of cable in total and no more than 2 metres between devices, and at least two-thirds of the devices powered on. Devices may be connected in a star, in a line, or in any combination of the two. See Figure 1–3.

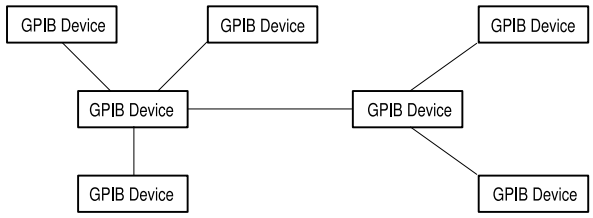


Figure 1–3: Typical GPIB network configurations

Setting the Address

Set the instrument's GPIB address from its front panel, and make a note of it. On a TDS 500, 600 or 700 series instrument, press **SHIFT**; then press **UTILITY**. Press the main-menu button **System** until **I/O** is highlighted in the pop-up menu, press the main-menu button **Port**, and select **GPIB**. The address is set from the same menu under **Configure**.

NOTE. *Set the instrument to Talk/Listen mode, not Off Bus or Hardcopy (Talk Only). In Talk Only mode it does not answer queries, and the program's scan will not find it.*

Installing the Program

From the Executable

Copy `TDS Toolkit.exe` to a folder you can write to and run it. The program is self-contained: it needs no Python, installs nothing and writes nothing to the registry.

The program keeps three items in the folder that holds the executable, and creates them there the first time it runs:

tdstoolkit.json — The settings: the language, the remembered instrument address, the plot colours, any colour schemes you have saved, and the saved-picture resolution.

tdstoolkit.log — A record of what the program did and of anything unexpected the instrument reported. Renamed to tdstoolkit.log.old when it reaches 512 kbytes.

masks — The mask library. The masks supplied with the program are copied here the first time it runs, and anything you draw and save joins them.

From Source

The program needs Python 3.8 or newer with Tkinter, which the Windows installer includes. Install the two packages it uses and run it:

```
pip install -r requirements.txt
python "TDS Toolkit.pyw"
```

`pyvisa` is the Python binding for the VISA runtime and is required. `tkinterdnd2` is optional and adds dragging files in from Explorer; without it everything else works and the title bar says so.

Starting the Program

The First Connection

Nothing needs configuring. The program tries GPIB0::3::INSTR, and if nothing answers it opens the instrument picker. The picker sends nothing on the bus until you press **Scan**; then every address VISA knows about is asked, in numerical order, to identify itself with *IDN?, and anything answering as a Tektronix TDS is marked as an oscilloscope. A known address is connected to directly, without a scan.

1. *Find the instrument:* Press **Scan**. If you already know the address, type it instead and skip the scan.
2. *Choose the instrument:* Click its row in the picker, or type an address into the **Address** box.
3. *Remember it:* Leave **Remember this address** ticked. The address is written to the settings file and the next run connects straight to it.
4. *Connect:* Press **Connect**, or double-click the row.

The instrument can be changed at any time from the **Instrument** dropdown above the tabs, which lists every TDS oscilloscope the last scan found. Its last entry, **Other address...**, opens the picker again. **Scan** beside it sweeps the bus; pressing it a second time while it runs cancels it.

The picker lists every address under **Instruments found on the bus:**, with what each one answered under **Identification** - the instrument's own `*IDN?` reply where it gave one, or the reason it gave none.

NOTE. *VISA lists every address it has ever been configured with, so a bus with one live instrument can show half a dozen. The picker shows why each address did not answer, so an instrument that is switched off can be told apart from one that was never there.*

The Window

The window is the same on every tab: the instrument dropdown, the **Scan** button and the language button along the top; the ten tabs below them; and a status line along the bottom, with a progress bar at its right while an operation is running. See Figure 1–4.

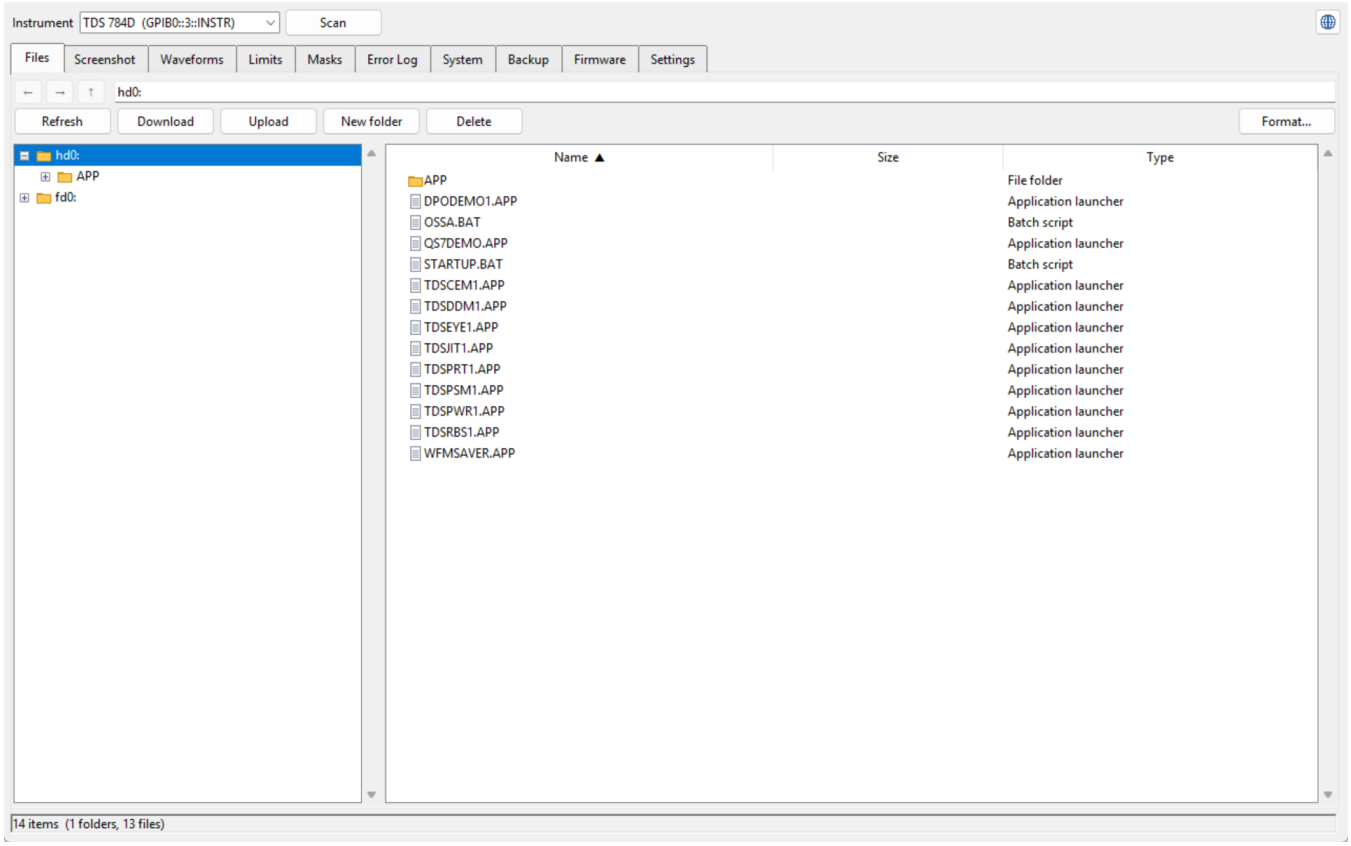


Figure 1–4: The program window, showing the Files tab

The status line is where the program reports. Every action puts a line there saying what it is doing, and another saying what happened. Anything unexpected that the instrument reports is shown there as well and written to the log.

While the Program Is Busy

One operation runs at a time. While the program is talking to the instrument the pointer becomes a watch and the buttons that would start another operation are greyed, on every tab, so that a transfer on one tab cannot be started on top of one on another. The drawing tools on the Masks and Limits tabs are not greyed, so drawing can continue while a transfer runs.

A button greyed for the whole session, rather than for the length of an operation, is one this firmware has no command for. The reason is given once, when the instrument connects.

Conventions

Throughout this manual the following conventions apply:

- The name of a control in the program is set in **bold**, exactly as the program labels it in English: **Get waveform**, **Send to instrument...**.
- The name of a control on the oscilloscope is set in bold and in capitals as it is engraved on the panel: **SHIFT**, **UTILITY**, **CLEAR MENU**.
- Commands sent over the bus are set in `Courier`, in the form the program sends them: `FILESYSTEM:READFILE`.
- A message the program shows is quoted in double quotes, word for word in English.
- Events the instrument raises are given by number and by the instrument's own wording: event 2241, "waveform requested is invalid".

Measurements quoted as measured were taken on a TDS 784D running firmware v7.4e, a TDS 784C running v5.3e or a TDS 640A running v3.8.8e, and the instrument is named where it matters.

Command Line

The program takes the following switches. An address given here overrides the remembered one for that run only and is not written to the settings file.

Table 1–3: Command line switches

Switch	Effect
--address=GPIB0::1::INSTR	Connect to this address instead of the remembered one. --address GPIB0::1::INSTR, with a space, is also accepted.
--version	Print the version and exit.
--selftest	Connect, probe the volumes, list a folder, read a file, write and verify a test file, and remove it. No window is opened. Use it to check the connection after changing the driver or cable.
--check-translations	Audit every language file beside the program and exit non-zero if any is wrong. No window is opened.

Files

The Files tab browses the instrument's drives and moves files either way. The program opens on this tab.

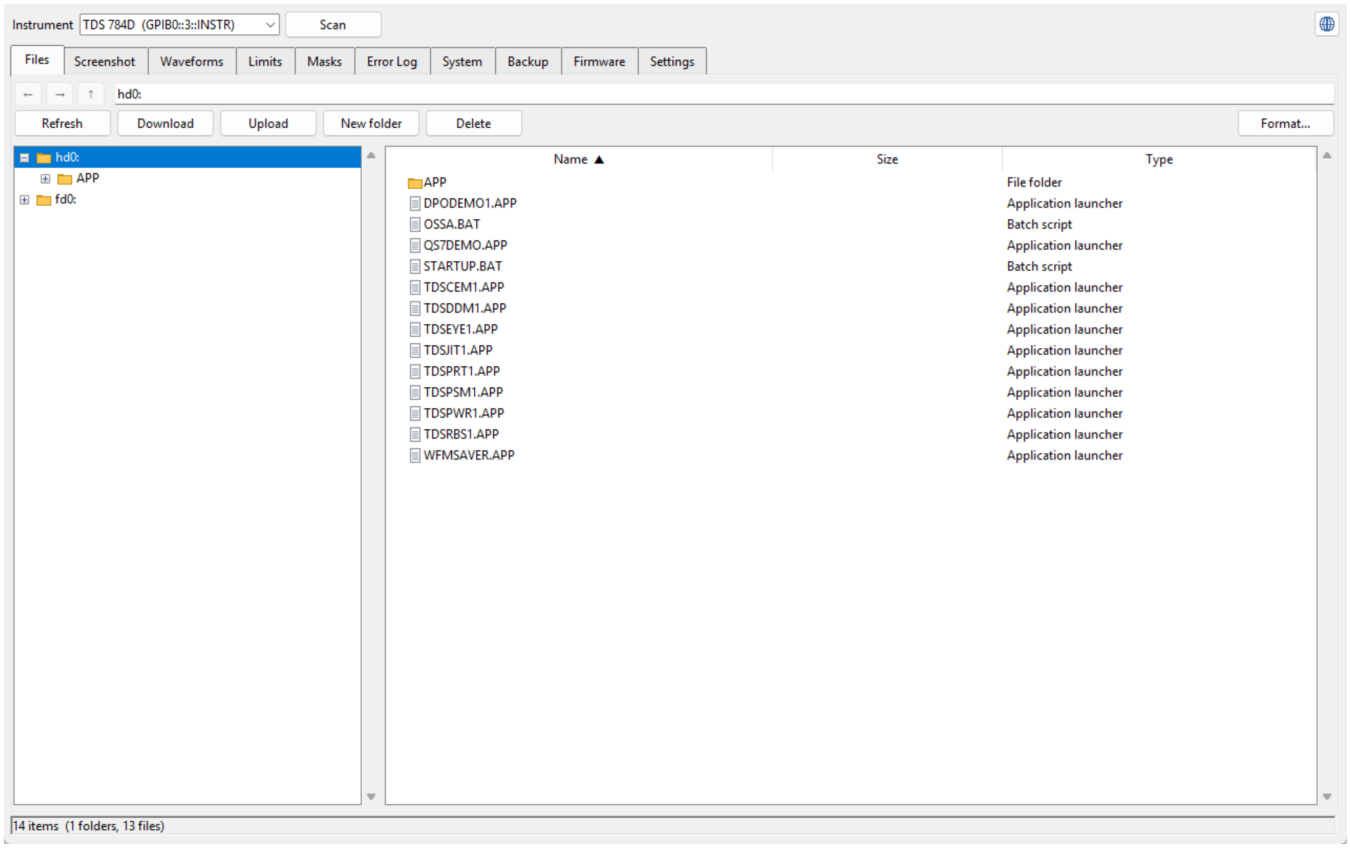


Figure 2–1: The Files tab

The Panes

Folder Tree

The left pane lists the drives and folders the instrument reports. The program probes for hd0:, fd0:, hd1: and fd1: when it connects, because the instrument has no command that lists its own volumes. It then opens hd0:, or the first volume found if there is no hd0:.

Click a folder to open it. A plus sign in a box beside a folder shows that it contains further folders and can be expanded; click the minus sign beside an expanded folder to close it.

File List

The right pane lists what is in the current folder. It has three columns:

Name The name as the instrument holds it, which is always in capitals.

Size In bytes, with thousands separated. Normally empty: the instrument's directory listing carries no sizes, so a size is known only after that file has been transferred, and is filled in then.

Type What the extension means. A folder reads "File folder"; an extension the program does not know reads "*EXT* File".

Click a column heading to sort, and click the same heading again to reverse the order. Folders sort above files whichever column is sorted. Sorting uses the listing already read; the instrument is not queried again.

NOTE. *Sizes are not in the instrument's directory listing and there is no command that returns one. The Size column is therefore filled in as files are transferred, and a folder you have only browsed shows none.*

What Is Not Listed

A card imaged from Windows carries VFAT long-filename records for any name Windows did not consider 8.3-clean. The instrument's FAT16 driver predates VFAT and reads each of those records as an ordinary directory entry, reporting it as a file with a two-character name and a trailing dot. Those entries are filtered out before the list is drawn.



CAUTION. *Do not attempt to delete such an entry by any other means. It is not a file: it is the record that holds the long name of the file beside it, and removing it strips the long name off that file.*

Moving About

The three arrows above the list are Back, Forward and Up, and the bar beside them is the directory path. Type a path and press Enter to navigate directly.

Table 2–1: Navigation keys

Key	Action
Alt+Left	Back to the folder before this one
Alt+Right	Forward again
Alt+Up, Backspace	Up one folder
F5	Read the current folder from the instrument again
Double-click, Enter	Open a folder, or save a file
Ctrl+A	Select every row
Delete	Delete what is selected

Back and Forward follow the browser convention: arriving at the folder you came from counts as a step back rather than a new destination, and going somewhere new discards the trail ahead of you.

NOTE. *Backspace and Delete act only while the Files tab is in front. The Alt+arrow keys and F5 work whichever tab is in front.*

How Long a Listing Takes

The instrument's `FILESYSTEM:DIR?` returns names with nothing to say which are folders, so each entry is classified by trying to change directory into it: two or three queries a name. The root of a hard disk holding 100 entries takes about three seconds. Each folder is listed once and remembered; **Refresh** reads it again.

Downloading Files

Select what you want and press **Download**, or double-click a single file. What happens next depends on the selection:

- **One file.** A Save As dialog opens with the instrument's name filled in.
- **Several files.** A folder is asked for, and each file is written as it arrives. Anything already there of the same name is listed and confirmed first.
- **One folder.** A folder is asked for, and the whole tree below it is downloaded into it. The tree is enumerated first, so the progress bar has a total to count against.

Folders are done one at a time. Selecting several and pressing **Download** raises "One folder at a time", because saving a folder brings down everything inside it.

Uploading Files

Press **Upload** and choose a file, or drag files onto the list from Explorer. The file is checked against the instrument's 8.3 name rule, written, and read back.

Whole folders can be dragged in as well, and go up complete: the folder, everything in it, and every folder below it to any depth. Folders are created on the instrument before the files that go in them. A folder name is held to the same 8.3 rule as a file name, so `My Captures` arrives as `MYCAPTUR`, and the drop plan lists every conversion before anything is sent.

NOTE. *A folder dropped onto a folder of the same name is merged into it, and the files that clash are replaced. That is how a folder already on the instrument is brought up to date. Names are kept unique within each folder rather than across the whole drop, so two files with the same name in two different folders are not a clash and neither is renamed.*

1. *Choose the destination:* Open the folder the file is to be written into. Uploads always go to the folder shown in the list.
2. *Choose the file:* Press **Upload** and pick it, or drag it onto the file list.
3. *Correct the name if it is refused:* A name that does not fit the 8.3 rule is refused with the reason, and a box opens with the name in it to be corrected. Names dragged in from Explorer are shortened automatically instead.
4. *Confirm:* The program states the file, its size and the destination. CHECK that they are what you intended, then press Yes.

Every upload is verified. The target is deleted, the file is written, the directory is re-read to confirm the name appeared, and the file is read back and compared byte for byte. Up to four attempts are made; a failure is reported as a failure rather than guessed at.

Names on the Instrument

The instrument's filesystem is FAT16 and its names follow the DOS 8.3 rule: up to eight characters, then optionally a dot and up to three more. Letters, digits and the punctuation `$ % ' - _ @ ~ ` ! ( ) { } ^ # &` may be used. The name is stored in capitals whatever case it is given in.

NOTE. *Dragging files in from Explorer converts the names rather than refusing them: the name is upper-cased, characters the instrument will not take are removed, and the stem is clipped to eight characters and the extension to three. Two names in one folder that come out the same after conversion are a clash, and the second is given a ~1 suffix. The drop plan lists every conversion before anything is written.*

NOTE. *A dropped name that matches a file already on the instrument replaces it. The confirmation lists what will be replaced and what will be merged into before anything is sent, so read it before pressing Yes - there is no undo on the instrument.*

How Long an Upload Takes

The payload crosses the bus twice, once to write and once to verify, and the two directions run at very different speeds. Measured on a TDS 784D over GPIB: writing to the hard disk runs at about 2.9 kbytes/s and reading back at about 32 kbytes/s, so the upload time is dominated by the write. A TDS 784C reads at about 3.9 kbytes/s and a TDS 640A at about 3.2 kbytes/s from a floppy. The drop plan estimates the time before anything is sent.

NOTE. *A 7 Mbyte folder therefore takes the better part of an hour, not a few minutes. Trust the estimate given in the confirmation, and let the program run rather than assume that a long upload has hung.*

Creating Folders

Press **New folder** and give it a name. The name is checked against the same 8.3 rule, checked against what is already in the folder, upper-cased, and sent as `FILESYSTEM:MKDIR`.

Deleting Files and Folders



WARNING. *There is no recycle bin on the instrument. A file deleted here is gone, and a folder is removed with everything inside it. Download anything you may want before deleting it.*

Select what you want to remove and press **Delete**. The pane that has focus decides what is deleted. Files and folders are deleted separately, and folders one at a time, so that the confirmation can show exactly what is about to be removed.

A folder is surveyed first, and the confirmation dialog names what is inside. The whole selection is checked before any of it is deleted, so if it includes any of the instrument's own files, the extra confirmation is asked once for the whole deletion rather than once per file.

The Instrument's Own Files

The instrument runs its applications from a Java runtime held on the same disk. These files can be deleted, but the program names them and asks a second time before it does so. Nothing on a floppy is treated this way.

Table 2–2: Files the program asks about twice

Name	Why
APP	Holds the Java runtime and every shipped application
APP/TDSRTE1 and everything below it	The Java runtime itself
STARTUP.BAT, OSSA.BAT, RTE1.BAT, RTE1ORIG.BAT	The instrument's boot chain. Deleting one prevents the runtime from starting at power-on
RT.JAR, JAVA68K.O, LIBJIT.O, NIGPIB.O, EXTCP.O, PATCH.O, LOGO.BIN, VERSION.DAT	Part of the Java runtime. Each of these loads back over GPIB, so a deletion can be undone

A drive itself cannot be deleted, and neither can the phantom entries that carry part of a long file name. Those two are refused outright rather than asked about.

Formatting a Volume



WARNING. *Formatting empties the whole volume. On the hard disk this includes the Java runtime and every application shipped with the instrument, and none of them will run until they are loaded back over GPIB. There is no undo.*

Press **Format...**, at the right-hand end of the button row. You are asked to confirm twice: the first names the volume and how much is on it, the second says what that volume holds. The program sends `FILESystem:FORMat` with the drive name quoted, then lists the volume again. The instrument reports nothing either way - the command has no query form and raises no event on success - so the new listing is the only evidence of success, and a volume that still holds files afterwards is reported as a failure.

Instruments With No Filesystem

Firmware v2.x has no filesystem commands at all: no working directory, no directory listing, no free space. On such an instrument the program says so when it connects and leaves the Files tab empty. The Waveforms tab still works, and on those instruments it is the main use of the program.

On the A and B series the filesystem is there but `FILESYSTEM:WRITEFILE` is not, so **Upload** is greyed for the session and the reason is given once. Browsing, downloading, creating folders and deleting all work normally.

What the Program Sends

The Files tab is a front end for the instrument's own filesystem commands. A folder is listed by changing into it with `FILESYSTEM:CWD` and asking `FILESYSTEM:DIR?`; the reply names what is in the folder but not which entries are folders, so each is classified by trying to change into it.

```
FILESYSTEM:CWD "<path>"       change into a folder
FILESYSTEM:CWD?               where the working directory is
FILESYSTEM:DIR?               list the current folder
FILESYSTEM:MKDIR "<path>"     make a folder
FILESYSTEM:READFILE "<path>"  read a file, bytes to EOI
FILESYSTEM:WRITEFILE "<p>",#0 write a file, then raw bytes
FILESYSTEM:DELETE "<path>"    delete a file
FILESYSTEM:RMDIR "<path>"     delete a folder, recursively
```

A download is `FILESYSTEM:READFILE` where the firmware has it. Where it does not - the A and B series - the program falls back to `FILESYSTEM:PRINT <path>,GPIB`, which is how those instruments hand a file over the bus.



CAUTION. *The PRINT fallback is tried at most twice for any one file. A long run of failed prints puts the instrument's filesystem out of action until it is power cycled, so the program stops rather than retrying.*

`DELETE` does nothing to a folder - no error, no event, just silence and the folder still there - so folders are removed with `RMDIR`, which is recursive, silent and has no dry run. Deletions are verified by re-listing the folder afterwards rather than by trusting an empty event queue.



CAUTION. *The program moves the working directory to the volume root before it sends RMDIR: if the working directory is inside the folder being removed, RMDIR is refused with event 257.*

Screenshot

A screenshot is taken through the instrument's hardcopy subsystem, which is designed to drive a printer but will send its output to the bus when asked. The capture is therefore a print job addressed to this program, which decodes it and draws it on the screen.

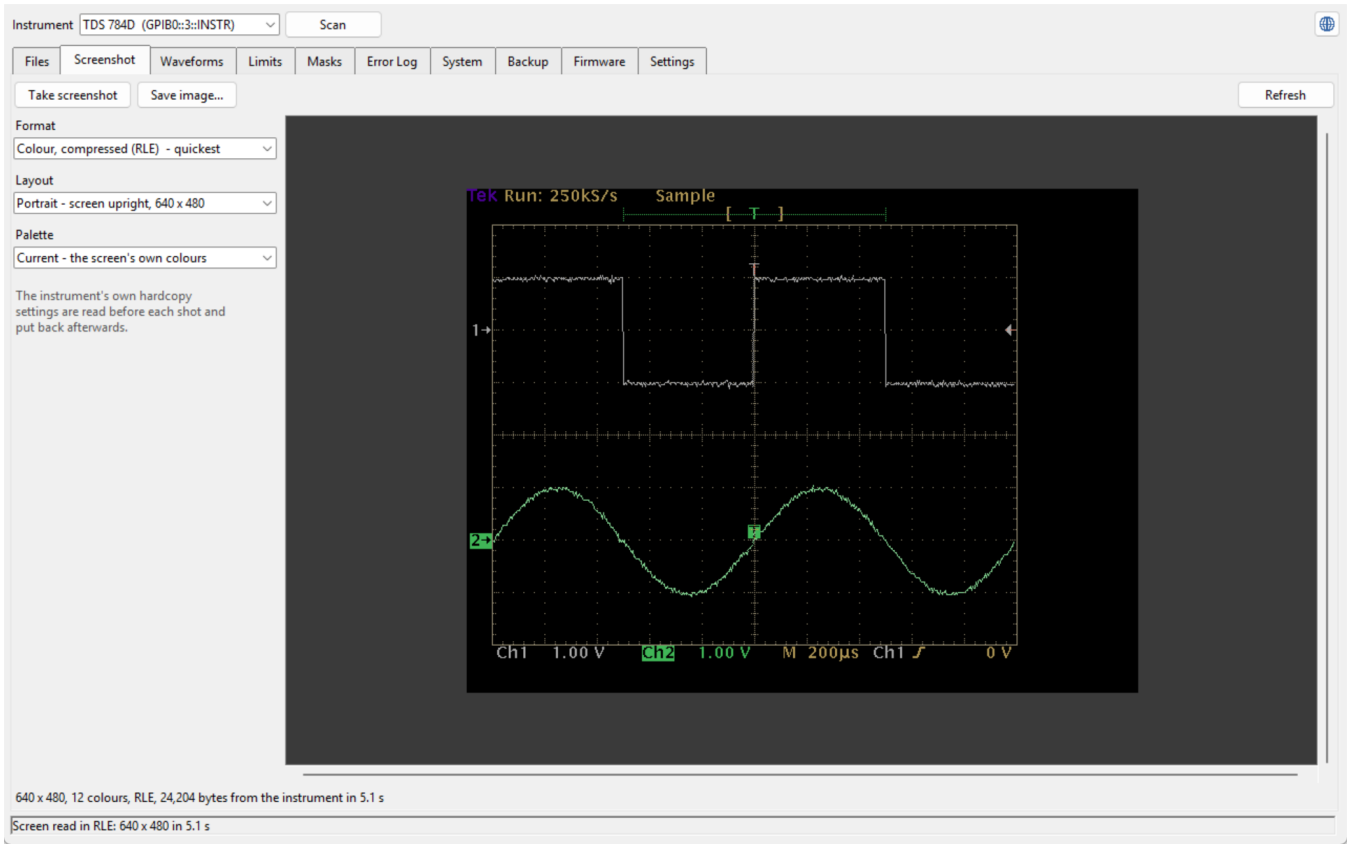


Figure 3–1: The Screenshot tab

Taking a Screenshot

1. *Choose the format:* The **Format** box lists only the formats this instrument accepted when queried. The best one available is already selected.
2. *Choose the layout:* **Portrait - screen upright, 640 x 480** gives the screen the right way up. **Landscape - screen turned 90 degrees** turns it on its side, at 480 x 640.
3. *Capture:* Press **Take screenshot**. The picture appears in the pane, and the line below it describes what was received.
4. *Save it:* Press **Save image...** and choose a name.

The line under the picture gives the size in pixels, the number of colours, the format the instrument used, how many bytes arrived and how long they took. The colour count is the number of palette entries the picture actually uses, not the size of the palette.

Formats

The instrument's hardcopy subsystem offers printer languages as well as image formats. Only the formats that are images, and that this program can decode, are listed.

Table 3–1: Image formats

Shown as	Keyword	Colour
Colour, compressed (RLE) - quickest	RLE	yes
Colour PCX	PCXCOLOR	yes
Colour, uncompressed (BMPCOLOR)	BMPCOLOR	yes
Monochrome (TIFF) - quickest	TIFF	no
Monochrome (BMP)	BMP	no
Monochrome PCX	PCX	no

Almost all of the time a capture takes is the instrument drawing the picture, so the smallest file is not the quickest. Measured over GPIB with a National Instruments GPIB-USB-HS:

Table 3–2: Measured capture size and time

Format	TDS 784D bytes	TDS 784D s	TDS 640A bytes	TDS 640A s
TIFF	38,830	3.28	38,858	3.27
BMP	38,462	4.35	38,462	6.14
PCX	17,763	4.40	14,587	6.43
PCXCOLOR	81,351	5.30	no colour formats	
RLE	85,318	5.35		
BMPCOLOR	308,278	5.62		

RLE is chosen by default where the firmware supports it: it is colour, it is a quarter of BMPCOLOR's bytes, and it is a Windows BMP with run-length encoding, so it needs no separate decoder.

On a monochrome instrument TIFF is chosen; it is twice as quick as BMP.

Layout and Palette

Layout

LAYOUT sets the orientation of the page, not of the image. Changing it rotates the picture already captured instead of taking a new capture.

A capture taken in LANDSCAPE arrives as a 480 x 640 image turned on its side; choosing **Portrait** turns it upright.

Palette

What the **Palette** box offers depends on the instrument. A colour instrument has a hardcopy palette of its own and is offered its two settings; a monochrome instrument has none, and is offered this program's own inversion instead.

Table 3–3: Palette choices

Instrument	Choice	What it does
Colour	Current - the screen's own colours	The instrument sends what the screen shows
Colour	Hardcopy - light background, for printing	The instrument sends its printing palette
Monochrome	Normal - dark on light, for printing	The picture as the instrument sends it
Monochrome	Inverted - as the screen shows it	Inverted by this program

NOTE. *On a colour instrument the palette belongs to the instrument, so the choice applies to the next capture and the picture on screen does not change. On a monochrome instrument the inversion is done in software, which avoids repeating a five-second capture.*

Saving the Picture

The file type list offers the instrument's own format first and PNG second. The image is saved exactly as shown in the preview pane.

NOTE. *A picture that is unchanged is saved in the native format exactly as sent from the instrument. One that has been rotated or inverted is written out again in the same format and at the same colour depth it arrived in. A picture that arrived compressed is written uncompressed.*

The suffix offered for the native format is taken from the bytes that arrived rather than from the format that was asked for, so an instrument that answers TIFF and sends BMP does not produce a mislabelled file. The default file name is stamped with the date and time, in the order year, month, day, so that a folder of them sorts chronologically.

What the Program Sends

The instrument's whole hardcopy configuration is read first, the capture is taken, and the original configuration is then written back.

```
HARDCOPY?                      (every field, read first)
HARDCOPY:PORT GPIB
HARDCOPY:FORMAT <keyword>
HARDCOPY:LAYOUT <layout>
HARDCOPY:PALETTE <palette>     (only where the field exists)
HARDCOPY START                 (then read until EOI)
```

Two delays are essential. A hardcopy started too soon after the previous one is ignored, with no data and no error, so two seconds are allowed to pass before START. The VISA timeout is raised to ninety seconds for the read itself.



CAUTION. *If a capture fails, the program issues a device clear and waits a second before reporting it. A half-sent hardcopy left in the instrument is what makes the next program's *IDN? time out.*

The picture's format is identified from its first bytes rather than from the format keyword. BMP at 1, 4 and 8 bits, uncompressed or run-length encoded, PCX at 1 or 8 bits, and the one baseline uncompressed form of TIFF a TDS writes are all handled. Anything else is refused by name. The instruments write a file-size field in the BMP header that is not the file's size - 77,070 in a 308,278-byte BMPCOLOR - and nothing in the decoder reads it.

Waveforms

The Waveforms tab captures what the instrument is acquiring, plots it, saves it, and writes it back. It works on every instrument in the range, including the v2.x models that have no filesystem at all, because the waveform transfer commands are present in every firmware image examined.

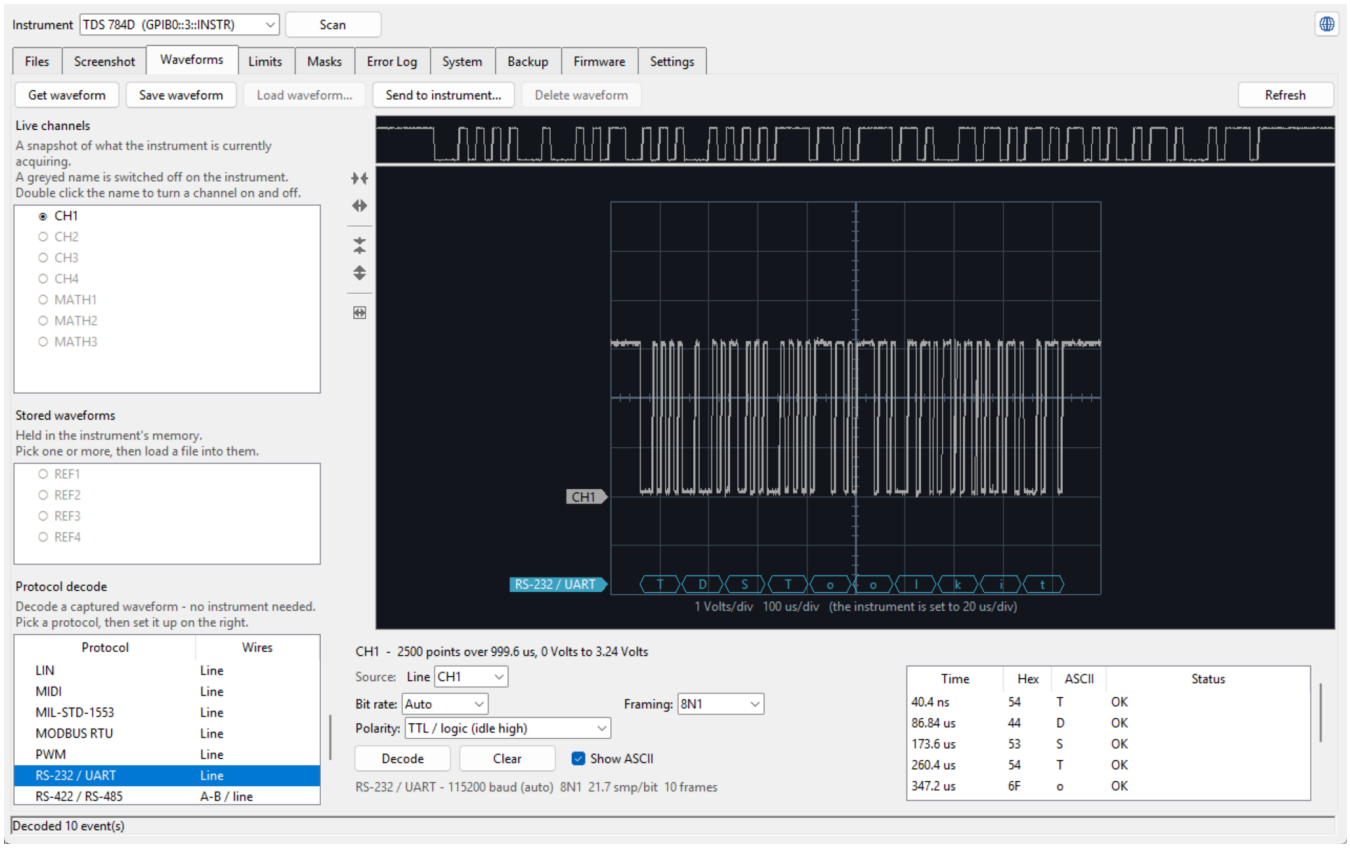


Figure 4–1: The Waveforms tab, decoding a captured RS-232 signal

The Source Lists

The instrument's sources are listed in two panes because they behave differently. A channel is acquired: every read is a fresh snapshot. A reference is stored: every read gives the same samples.

Live channels Every channel and maths waveform the instrument reports. A name in grey is switched off at the instrument.

Stored waveforms The reference memories the instrument reports, normally REF1 to REF4.

To choose a source, click the symbol at the left of its row, which then fills in. Double-clicking a name switches that source on or off at the instrument.

NOTE. *A channel that is not displayed cannot be read. Asked for one, the instrument answers event 2241, "waveform requested is invalid", and follows it with a query interrupt. Choosing a greyed name therefore switches it on at the instrument first. A stored reference is the exception and reads whether it is displayed or not.*

Capturing

Press **Get waveform**. Every live source that is chosen and displayed is read from one acquisition, and any chosen references are read after it.

The acquisition is stopped for the read and released afterwards. This is essential. Reading CH1 and then CH2 while the instrument is running gets two different captures: measured on a 784D triggering thirteen times a second, the two reads were 600 ms apart and disagreed about XZERO. With the acquisition stopped, the two channels report the same XZERO, and the same source read twice comes back byte for byte the same.

Three things are deliberately not done:

- An instrument that was already stopped is left stopped.
- An instrument set to single sequence, or set to stop on a limit test, is not touched: stopping it would re-arm the trigger, and with STOPAFTER LIMIT there is nothing for an operation-complete query to complete, so the bus would wait out its timeout.
- References are read after the acquisition is released, since their contents do not change.

The status line says which of the three happened, for example "Captured CH1, CH2 in 0.85 s - acquisition held for the capture, then released".



WARNING. *Waveform capture is refused while the instrument is in DPO, and the program does not switch DPO off. In DPO the instrument builds a pixel database instead of filling a waveform record, and every point of every channel comes back as 128. Switching DPO off would discard the accumulation on the screen, which is the very thing being looked at. Take a screenshot instead, or disable DPO mode yourself.*

Reading the Plot

Traces are drawn at the instrument's own scale: each sample is plotted at its digitizer count divided by the counts per division, which is exactly where the instrument puts it on its own screen. A small signal on a large volts-per-division setting therefore draws small, the way it looks on the scope, rather than being stretched to fill the graticule.

Record strip The band above the graticule shows the whole record with the visible window outlined, the way the instrument's own record view does.

Channel marker The arrow outside the graticule sits at that trace's zero volts and carries its name. Markers that would collide step aside, and each points towards the graticule.

Caption Below the graticule: the volts per division and seconds per division of the view. Where the view is longer than the instrument's ten divisions, the instrument's own setting is given as well.

Table 4–1: Mouse and keyboard on the plot

Gesture	Effect
Drag	Pan. Whatever is under the pointer stays under it
Drag a marker	Move that trace only, up and down, like the instrument's position knob. It snaps onto the centre line within half a minor division
Shift and drag	Rubber band. Zoom to the rectangle on release
Wheel	Zoom in time about the pointer
Shift and wheel	Zoom in amplitude about the pointer
Click or drag the record strip	Move the window. It centres on the pointer and follows it
Drag an edge of the marked window	Stretch the window from that edge. The other end stays where it is, and the pointer changes shape within five pixels of an edge
Wheel over the record strip	Zoom in time about the moment under the pointer
Right-click a marker	Hide that trace, or show only that trace

Zoom and Pan

The zoom buttons to the left of the plot step the time and amplitude scales through a 1-2-5 sequence, the same one the instrument's own knobs use. **Whole record**, the last of them, puts everything back.

A window stretched on the record strip is rounded to a whole number of microseconds per division, so the same drag always gives the same scale. Below one microsecond per division the 1-2-5 sequence is used instead, because whole microseconds no longer divide the view. Either way, rounding is limited by the room available: an edge dragged to the end of the record gives exactly the span asked for rather than a round number, because the held end does not move.

Saving a Waveform

Save waveform writes the current trace. The format follows the extension:

Table 4–2: What each format keeps

Extension	What is written
.csv	A header line naming the units, then scaled x,y pairs. Read back it is re-quantised, and is correct to within one step of the instrument's eight-bit range
.isf	The instrument's own transfer format: the preamble fields and the curve exactly as CURVE? returned them. Read back it is byte for byte the same
.tdw and .wfm	The instrument's own file layout, saved under this program's extension or under the instrument's own. The bytes are the same either way, and eight-bit captures are widened to sixteen bits. See the note below
.png	The graticule as it is on screen now, at the current zoom and with the record strip above it, at the resolution set on the Settings tab and in the current colours
.svg	The same picture as .png, drawn as scalable vector so it stays crisp at any size and can be opened in a vector editor

NOTE. *`.tdw` is this program's own extension for the instrument's layout, kept so that a file this program wrote can be told from one an instrument wrote. Both are written and both are read; choose* `.wfm` *where the file is going to an instrument or to another tool, and* `.tdw` *where it is not. To write one straight to the instrument's own disk, use* ***Send to instrument...***

NOTE. *The .WFM format is not documented by Tektronix. What is written here was worked out by comparing the instrument's own files against Tektronix's CNVRTWFM converter, one field at a time, and every field the program reads and writes agrees with that converter on every file measured. The rest of the 132-byte header was settled by comparing many files the instrument wrote: nearly all of it is the same in every one — a reserved word at the front and the display's zoom and pan state, which the instrument leaves at its defaults — and the program writes those back unchanged. Only a short run near the end of the header differs from one waveform to the next; it holds what looks like a per-waveform checksum and a pair of small counters. These are carried over as the instrument wrote them, and they fall inside the file's own end checksum, which the program recomputes, so a written file is self-consistent either way.*

Loading a Waveform

A file is loaded into a reference, and nothing is written to the instrument until **Send to instrument...** is pressed. Several references can therefore be filled at once, and a file can be inspected without committing it to the instrument.

1. *Choose the reference:* Click the button beside REF1, or beside several references at once.
2. *Open the file:* Press **Load waveform...** and choose an ISF, CSV, TDW or WFM file. The row then reads, for example, "REF1 - trace.isf, not sent yet".
3. *Send it:* Press **Send to instrument...** and confirm.

The file's type is established from its contents rather than from its extension. A waveform file from a TDS5000, TDS6000, TDS7000 or a DPO or MSO is a different and much later format and is refused by name rather than misread.

Sending a Waveform to the Instrument

Send to instrument... writes the current trace into a reference memory, or onto the instrument's disk where the firmware has WRITEFILE. You are asked for the destination:

- **Reference memory REF*n*.** The waveform is written into that reference and read back to verify it.
- **A .WFM in *drive*/WAVEFORM, which the instrument can recall.** Always a folder called WAVEFORM at the root of the drive the Files tab is on, created if it does not already exist. The name is four characters of the source and a four-digit serial - CH10001.WFM, CH10002.WFM - and the serial is the first the folder does not already hold, so a second send keeps the first rather than replacing it. The write is read back and compared like any other upload.

Only the instrument's own format goes onto its disk. An .ISF written there is a file the instrument itself can do nothing with, and copying one back to a computer over GPIB is a wasted round trip when **Save waveform** writes the same file straight to the computer's own disk.



WARNING. *Whatever the destination reference holds now is replaced. There is no undo, and the instrument gives no warning of its own.*

Every send is read back and compared. A mismatch is reported as "*REF1* was written but reading it back gave something else - the instrument did not keep what was sent."

Deleting a Reference

Delete waveform empties the chosen reference. Only a stored reference can be deleted; a channel or maths trace is computed live and has nothing stored to remove, which is why the button is greyed unless a reference is chosen.

Protocol decode

A captured waveform can be decoded against a serial or parallel protocol without involving the instrument. The decode runs on the samples the program already holds - a channel that was read, or a reference loaded from a file - which is what lets it work on instruments whose firmware cannot run a bus-decode application of their own. That application needs a Java runtime the base models do not carry; this decode needs only the samples.

The protocols are listed under the two source panes. Choosing one fills the panel below the plot with a source box for each of its wires and a control for each of its settings; map a captured source to each wire and press Decode. The decoded events are drawn as chevrons along the trace, in a band that can be dragged up or down and is labelled with the protocol it decodes, and are listed in the table beside the controls. A source can be captured on one instrument and decoded against a protocol driven from another - the decode cares only about the samples.

Twenty-six protocols are supplied:

Async serial	RS-232 / UART, RS-422 / RS-485, MODBUS RTU, MIDI, LIN, XBee, DMX512, CRSF, SBUS, IrDA and ISO 7816.
Clocked	I2C, I3C, SPI, Dual SPI and I2S.
Addressable LEDs	WS2812 / SK6812 and APA102 / DotStar.
Remote control	IR, with thirteen protocols: NEC and NEC extended, Sony SIRC, Philips RC5 / RC5x / RC6, Panasonic Kaseikyo, RC-MM, and Samsung, LG, JVC, Denon and Mitsubishi.
One-wire and edge	1-Wire and PWM.
Automotive	CAN, LIN and FlexRay.
Avionics and audio	ARINC 429, MIL-STD-1553, S/PDIF and I2S.

Adding a protocol

Each protocol is one Python file in a decoders folder beside the program's settings. Like the mask library, the folder is filled from the bundled set and topped up from it on every run, so new decoders from an update arrive without overwriting any you have written. A file dropped into that folder appears in the list the next time the tab is opened; one that will not import is skipped with a note in the log instead of breaking the list.

NOTE. *A decoder carries its own chevron colours - one for a clean event and one for a fault - so several decoded buses on one screen can be told apart, and it can colour each kind of event in the line differently. There is no colour setting for decoders on the Settings tab.*

What the Program Sends

Reading one source:

```
DATA:SOURCE <source>
DATA:ENCDG RIBINARY
DATA:WIDTH <1 or 2>
DATA:START 1
DATA:STOP 1000000
HEADER ON
WFMPRE:<source>?              (the whole preamble at once)
HEADER OFF
DATA:START 1
DATA:STOP <NR_PT>
CURVE?
```

The order of two steps in that sequence matters:

- DATA:SOURCE must be written before any WFMPRE query, or the instrument answers event 2241 even for a reference that demonstrably holds a waveform.
- NR_PT does not report the record length. It reports how many points would be sent, which is DATA:START to DATA:STOP. A 784D was measured reporting NR_PT 20 for a 5000-point record, because something earlier had set DATA:STOP 20.

The preamble is asked for with the headers on, so that every field names itself. The headers-off reply is not in the same field order on every instrument, and mapping one instrument's order onto another puts a unit string where a sample count belongs. Named, the order does not matter. Compared with asking for the eleven fields one at a time, this cuts 419 ms to 81 ms on a 784D, 457 ms to 76 ms on a 784C, and 3510 ms to 196 ms on a 640A, which has no XZERO and does not answer the query for it at all.

NOTE. *Where the bulk reply comes back in some shape the program does not recognise, it falls back to asking the eleven fields one at a time. If one instrument in thirty-four answers unexpectedly, it should cost a little speed, not the capture.*

Limits

A limit test judges every acquisition against an upper and a lower limit and stops the instrument the moment the signal leaves them. The instrument does the judging; this tab builds the template, starts the test and reports the verdict.

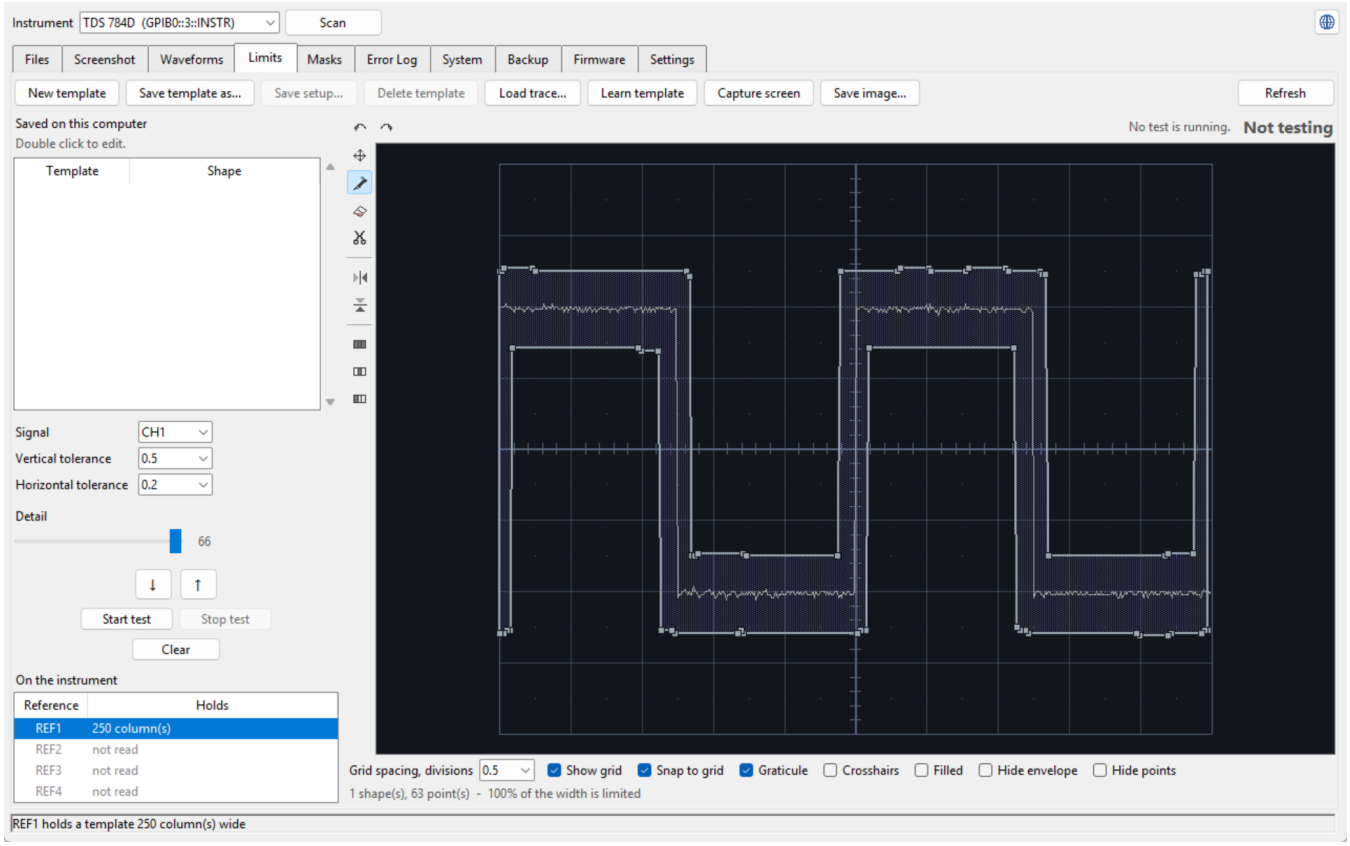


Figure 5–1: The Limits tab, showing a template learnt from a square wave

Running a Limit Test

1. *Display the signal:* Set up the instrument to display a signal known to be good. The template is built from this acquisition, so everything the test judges is judged against it.

2. *Choose the signal and the destination:* Set **Signal** to the channel to be judged, and click the reference the template is to be written into in the **On the instrument** list. The selected row is the destination for everything on this tab, and the list says what each of the four references already holds.
3. *Set the tolerances:* Enter how far the signal may wander, vertically and horizontally, in divisions of the graticule.
4. *Make the template:* Press **Learn template**. If no trace is loaded, it offers to have the instrument build the template from the displayed signal, and the tab then draws it.
5. *Start the test:* Press **Start test**. The verdict reads PASS while the instrument is running.
6. *Stop the test:* Press **Stop test** to switch the test off and let the instrument run freely again.



WARNING. *Whatever the selected reference holds now is replaced, both by* ***Learn template*** *and by the down arrow. Save anything you need from that reference first.*

NOTE. *When building from the instrument,* ***Learn template*** *does more than read the signal: the instrument builds a template from it and stores that in the selected reference. Use* ***Refresh*** *to look at the signal and the template without writing anything.*

Learning From a Loaded Trace

If **Load trace...** is used first, **Learn template** builds the envelope around that trace instead of fetching another one from the instrument. It is the same shape the instrument would build - each sample may arrive as early or as late as the horizontal tolerance allows, and sit as high or as low as the vertical tolerance allows - but it is worked out here.

Nothing is written to the instrument, and none is needed: the envelope appears on the graticule as a drawing, ready to be adjusted, and the down arrow is what puts it into the reference. A template can therefore be built from a previously saved waveform on a computer with no instrument attached at all.

NOTE. *Only a trace placed with* ***Load trace...*** *counts. The one* ***Refresh*** *reads back is the instrument's own record, and learning from that would only repeat the instrument's own Learn with an extra round trip.*

The Toolbar

These are the same buttons as on the Masks tab, and they do the same things to a template. **New template** starts again on an empty graticule - the drawing, the template read out of the reference, the trace and any captured screen are all cleared, and the instrument keeps whatever it is holding. **Save template as...** and **Delete template** work on the library described below, and **Save setup...** writes the instrument's settings beside the template's own file.

Load trace... puts a captured trace or a saved waveform on the graticule to draw against, and **Capture screen** puts the instrument's own display behind the envelope - which is the only picture available when the instrument is in DPO. **Save image...** saves the graticule, everything on it included, and is greyed while there is nothing on it to save.

The Reference List

The four references, and what each one holds. Opening the tab reads nothing: every row starts greyed and reads "not read", and a single click selects a reference as the destination without going near the instrument. Double-clicking a row reads that one reference, which then reads either the number of columns its template has or "empty", in ordinary type. A reference found empty is reported on the status line, not in a dialog: an unused reference is the ordinary state of one, not a fault.

The Buttons Beside the Reference List

All the controls that act on the instrument's references sit beside that list, just as the Masks tab keeps its transfers beside the segments they move. The two arrows are the two directions: down writes what is drawn here into the selected reference, up reads back what that reference already holds and puts it on the canvas to be looked at or adjusted. **Clear** deletes what the reference holds on the instrument, which cannot be undone and is refused while a test is running.

NOTE. *The up arrow replaces what is drawn, and asks first when there is a drawing to lose. One press of undo puts it back.*

The Template Library

Saved on this computer lists the limit templates kept beside the program, in the way the Masks tab lists masks. **Save template as...** writes what is drawn to a .LIM file, double-clicking a row puts that template back on the canvas, and **Delete template** removes it from this computer without touching the instrument. A .LIM file is a mask file carrying one marker line, so the same reader opens both and neither tab mistakes one for the other.

Tolerances

Vertical tolerance is how far the signal may wander up and down, and **Horizontal tolerance** how far it may move in time. Both are in divisions of the graticule, from 0 to 5, which is the range the instrument's own command takes. Half a division is 0.5. A tolerance of zero is accepted and gives a template exactly as wide as the signal itself.

NOTE. *The tolerance is applied by the instrument when it builds the template, not by this program. Changing a tolerance after a template has been built has no effect until it is built again.*

The Verdict

The instrument reports a failure by stopping. That is the whole of the verdict, and it is what this tab watches for: while the acquisition is running the test is passing, and the moment it stops the signal has left the template. The tab polls every two seconds, and polling writes nothing to the instrument.

Table 5–1: What the verdict means

Verdict	State
Not testing	No test is running
PASS	The signal is inside the template and the instrument is still acquiring
FAIL	The instrument stopped, so the signal left the template

Drawing the Envelope

The envelope the signal must stay inside can be drawn by hand instead of being built by the instrument, and a built one can be adjusted. The editor is the same one the Masks tab uses and is described in Section 6; the difference is what the drawing means.



CAUTION. *On this tab the envelope is where the signal may go. On the Masks tab the same shape means the opposite: where the signal may not go. Sending one in place of the other lets through everything it was drawn to catch.*

The down arrow beside the reference list writes what is drawn into the selected reference. **New template** clears the graticule and leaves what the instrument is holding alone.

A limit test allows the signal one band. A drawing that leaves two bands in a column, with a gap between them, cannot be expressed, and the program says which columns and offers to close the gap up.

NOTE. *A drawing on this tab may hold up to 64 shapes of up to 200 points each. Those are not instrument limits: the drawing becomes one envelope, as wide as the record is long, whatever it is made of. They are there to guard against a runaway drawing, and to keep the handles to a number a person can work with.*

What a New Template Leaves on the Canvas

A template holds two values for every column of the record, so a 500-point record gives 250 columns and five hundred handles, which is far more than anyone can edit. The program thins it to as many handles as the **Detail** slider asks for - 120 to begin with, half to each edge, and anything from 20 to 500 - and leaves the instrument's own band drawn underneath so the cost of the thinning can be seen. Dragging the slider re-thins from that band rather than from the drawing, so the choice can be seen rather than guessed at. The thinning only ever loosens. Thinned without that rule, the upper edge would cut every corner inwards and take away exactly the margin the tolerance was meant to give.

What the Program Sends

```
ACQUIRE:STOPAFTER RUNSTOP
LIMIT:STATE OFF
DISPLAY:MODE NORMAL
ACQUIRE:STATE RUN
*OPC?
LIMIT:TEMPLATE:SOURCE <source>
LIMIT:TEMPLATE:DESTINATION <dest>
LIMIT:TEMPLATE:TOLERANCE:VERTICAL <divisions>
LIMIT:TEMPLATE:TOLERANCE:HORIZONTAL <divisions>
LIMIT:TEMPLATE STORE
```

Learn Template

The tolerances are set before STORE, because STORE is what reads them. The display is taken out of DPO first: a limit test under InstaVu reads back as on and tests nothing, answering event 550, "InstaVu active", and a template built there would be built from a record that does not exist.

The result is a true envelope. Measured on a 784D, the reference reported "Ref3, DC coupling, 500.0mVolts/div, 200.0ns/div, 500 points, Envelope mode", and a test against it held while the signal was unchanged and stopped within two seconds of the amplitude being doubled.

```
ACQUIRE:STOPAFTER RUNSTOP
DISPLAY:MODE NORMAL
ACQUIRE:STATE RUN
*OPC?
LIMIT:COMPARE:<source> <dest>
LIMIT:STATE ON
*OPC?
LIMIT:STATE?                 (read back to confirm)
ACQUIRE:STOPAFTER LIMIT
ACQUIRE:STATE RUN            (and no *OPC? after it)
```



CAUTION. *Nothing may ask *OPC? after ACQUIRE:STOPAFTER LIMIT. With the acquisition told to stop only when the test trips, there is nothing for an operation-complete query to complete, and the bus waits out its timeout.*

Start Test

LIMIT:STATE is read back rather than assumed. An instrument that will not accept the test shows this by leaving the state off, and its own explanation is drained from the event queue and shown.

Reading the Template Back

Refresh reads the template out of the reference and the live signal off the channel, and draws both. Two orderings matter, and both were measured:

- The preamble must be read after DATA:WIDTH 2, because WFMPRE answers for the width currently set. Read first, YMULT comes back as the eight-bit 20 mV a count rather than the sixteen-bit 78.125 uV, and every volt in the band is 256 times too big, which draws a band covering the whole graticule. A band that covers everything passes everything.

- DATA:STOP counts values, not points. Asked for twice NR_PT the instrument answers event 531, "Data stop > record length, Curve truncated", and sends NR_PT of them anyway, giving half the columns spread over the whole record. Measured against a 1 kHz square at 200 us/div, the edges landed 62.3 columns apart out of 250, which is 498 us where 500 is right.

Sending a Drawn Envelope

A drawn envelope becomes an upper and a lower value for each column, loaded into the reference as an envelope. The scaling fields are copied from the channel's own preamble, because a template written against a 1 V/div channel came back as 500 mV/div otherwise, and because a channel sitting at YOFF 12800 needs the band built about 12800 or it lands two divisions out and fails a good signal.

NOTE. *Reading an envelope back from an instrument has not yet been tested on real hardware. The program reports a mismatch on the status line rather than raising it as an error, because the read-back itself is not trusted enough to call the instrument wrong.*

Masks

A mask is a keep-out area drawn in percent of the graticule. The instrument holds eight of them, draws them on its screen, and counts how many acquisitions touch them. This tab keeps a library of masks on the computer, edits them, sends them to the instrument, and sets the instrument up to judge a signal against one.

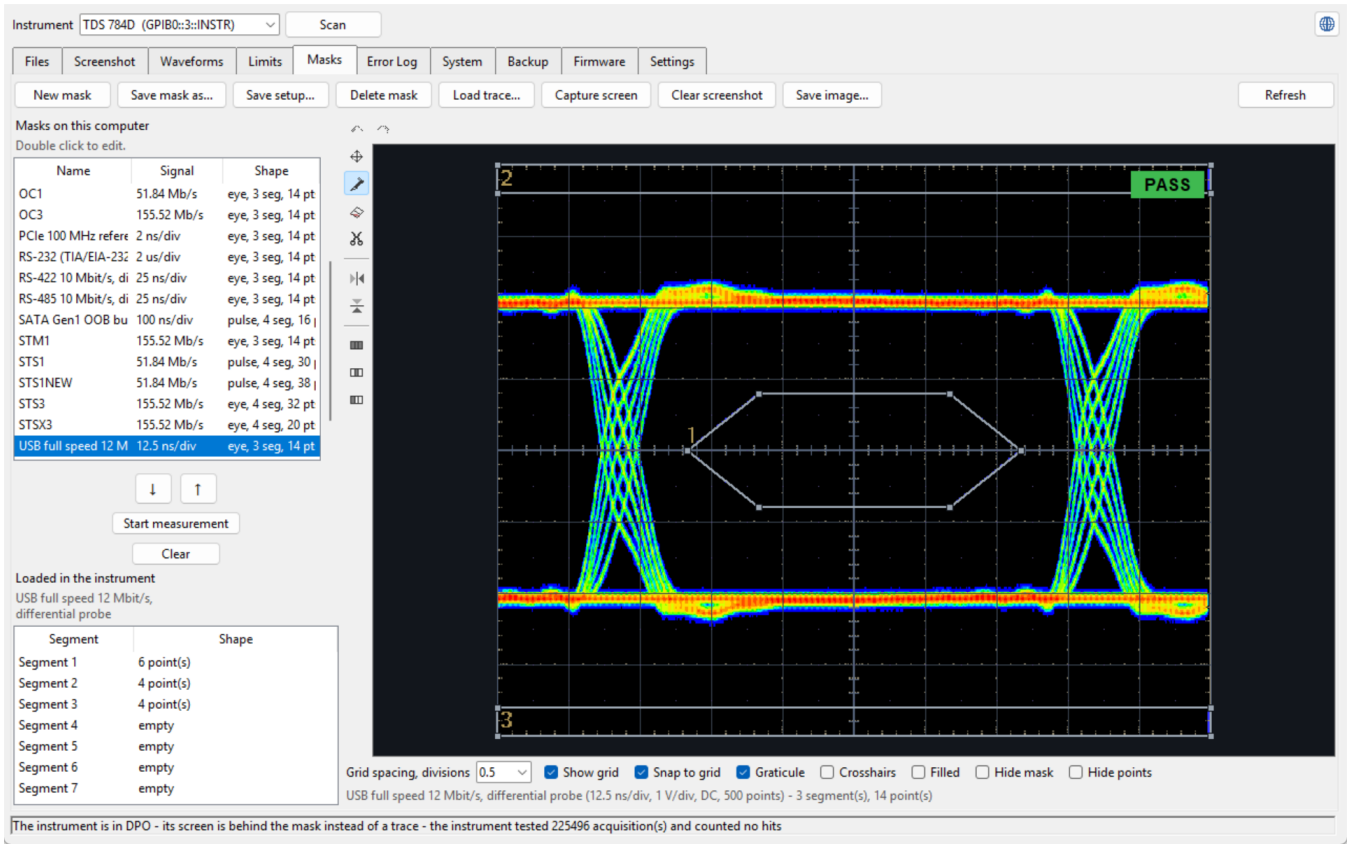


Figure 6–1: The Masks tab, with a USB full speed mask open

NOTE. *The mask subsystem is Option 2C. Without it the instrument answers the mask queries and draws nothing, so* ***Send to instrument*** *and* ***Start measurement*** *are greyed for the session. Drawing, editing, saving and loading are not: they need no instrument at all. To judge a signal on an instrument with no 2C, use the Limits tab, which reaches a verdict by a different means. See Section 5.*

The Toolbar

New mask starts again on an empty graticule: the drawing, any trace loaded behind it and any captured screen are all cleared, and the instrument keeps whatever it is holding. **Save mask as...** writes what is on the canvas into the library, and **Delete mask** removes the selected entry from this computer. **Save setup...** writes the instrument's settings beside the mask's own file - see *Mask Files* at the end of this section.

Load trace... puts a captured trace or a saved waveform on the graticule to draw against. **Capture screen** puts the instrument's own display behind the mask, which is the only picture there is when it is in DPO, and **Clear screenshot** takes that picture away again. **Save image...** saves the graticule with everything on it, and is greyed while there is nothing to save. **Refresh**, at the right-hand end, reads the library again and asks the instrument what its eight segments hold.

The Library

The upper list on the left is the library of masks on the computer, kept in a `masks` folder beside the program. The masks supplied with the program are copied there the first time it runs. Double-click a row to open that mask in the editor.

Table 6–1: What the library lists

Column	What it shows
Name	The mask's own name, from inside the file
Signal	The standard's bit rate where the mask is one Tektronix shipped, otherwise the timebase from the setup beside it
Shape	Whether it is a pulse mask or an eye mask, and how many segments and points it holds

Loaded in the instrument, the lower list, shows the eight segments the instrument itself is holding, and the line above it names what was last sent from here. Between the two lists are the controls that move a mask either way.

Down arrow Send the open mask to the instrument's eight segments.

Up arrow Read the instrument's eight segments back into the editor.

Start measurement Set the instrument up to judge this mask and start counting.

Clear Empty the instrument's eight mask segments.

NOTE. *The instrument has no mask files: there is no SAVE:MASK and no RECALL:MASK. The eight segments are all the instrument holds, and the library lives on the computer.*

Drawing

Coordinates are in percent of the graticule, X across and Y up from the bottom, so the whole graticule is 0 to 100 in both. One division is 10 percent of the width and 12.5 percent of the height, which is why the grid steps differ in the two axes.

The Tools

The tools are a column of buttons to the left of the canvas, shown as icons and named only by the tooltip that appears when the pointer rests on one. They are listed here in the order they appear, top to bottom. The first four are modes, and exactly one is active at a time; the rest act at once on what is selected.

Table 6–2: The tool palette

Icon	Named	What it does
Four-way arrow	Select and move points and shapes	Drag a point to move it, or an edge to move the whole shape. Arrow keys nudge what is selected
Pen	Draw points	Click to place points. Clicking the first point again closes the shape; a double-click, a right-click, Enter or Escape also finish it
Eraser	Delete points and shapes	Click a point to delete it, or an edge to delete the whole shape
Scissors	Split a shape between two of its points	Click one point of a shape, then another, to cut it in two between them
Two wedges	Mirror the selection horizontally, or vertically	Flip what is selected about its own axis
Two squares	Union, Intersect, Subtract	Combine two chosen shapes. The order matters: subtract takes the second out of the first

Choose a shape with the move tool, then hold Shift and click a second one to choose both. The status line under the canvas always says what the active tool does.

Table 6–3: Keys in the editor

Key	Action
Escape, Enter	Finish the shape being drawn, or abandon a cut
Delete, Backspace	Delete the selected points. Two neighbouring points of one shape collapse to their midpoint
Arrow keys	Nudge the selection by a fifth of the grid step, or by the distance set on the Settings tab
Ctrl+C, Ctrl+X, Ctrl+V	Copy, cut and paste whole shapes
Ctrl+Z, Ctrl+Y	Undo and redo, twenty deep

NOTE. *Y counts upwards here: percent is measured from the bottom of the graticule. The instrument numbers its own mask points from the top, and the conversion between the two happens once, when the mask is sent.*

Typing Coordinates

Double-click a point to open the Coordinates dialog for its shape. The dialog lists every point with its X and Y to two decimal places, and carries the shape's midpoint at the top under **Shape centre**. Typing into either moves the shape or the point as you type, and the whole edit is one undo step. While the dialog is open the points are numbered on the canvas.

Right-clicking opens the same dialog, and is the way to reach it for a whole shape rather than a point. Right-click a point for **Delete** and **Coordinates** for that point; right-click a side or the inside of a shape for the same two, acting on the shape. A shape's coordinates are its bounding box's middle, so typing 50, 50 there puts the shape in the middle of the graticule.

NOTE. *While a shape is being drawn, neither of those happens: a double-click or a right-click finishes the shape instead. The second click of a double-click lands on the point the first one laid down, which is not a request to edit it.*

Options Under the Canvas

Grid spacing, divisions	The grid step, in divisions.
Show grid	Draw a dot at every grid crossing.
Snap to grid	Whether clicks, drags and nudges snap. Independent of Show grid.
Graticule	Draw the instrument's 10 by 8 graticule.
Crosshairs	Lines that follow the pointer, with a readout.
Filled	Draw the shapes solid rather than outlined.
Hide mask	Hide the shapes so the trace behind can be seen. Turned off again whenever a tool is selected.
Hide points	Draw the shapes without their handles, for looking at the shape rather than at what it is made of. Turned off again whenever a tool is selected.

What the Instrument Will Take

The instrument's mask subsystem has hard limits, and a mask that breaks one cannot be saved or sent until it is corrected:

Table 6–4: Mask limits

Limit	Value
Segments	8
Points on segment	50
Minimum points on segment	2. A single point comes back as 0,0
Coordinates	0 to 100 percent in both axes

The instrument also stores the points exactly but joins them up in its own way. Three arrangements come back drawn differently from the way they went in, and the program warns about each before sending:

- Three points in a line one above the other: the instrument drops one of them.
- A shape a vertical line can cross more than twice.
- A shape a horizontal line can cross more than twice.

Concavity on its own is not a problem. Cutting the shape into simpler pieces avoids all three, and a segment in this state is drawn dashed on the canvas.

Sending a Mask

Press the down arrow. The dialog that opens states where the mask is going and what will be replaced.



WARNING. *Sending replaces whatever mask the instrument is holding, all eight segments of it. What is on the computer is untouched.*

Where a setup file sits beside the mask, the program offers to send that first. The setup names the channel, the volts per division and the timebase the mask was drawn against, and without it the mask is judged against whatever the instrument happened to be set to.

A setup is read back after it is sent and compared with what was asked for. Where the instrument silently set something else, the program says which commands and offers to stretch the mask to mean what it meant before, because a mask is held in percent of the graticule and can be stretched to match. Where the scale asked for was out of the channel's reach, it says which probe would reach it.

Judging a Signal

Capture screen puts what the instrument is showing behind the mask: a trace where it can be read, or the instrument's own screen where the instrument is in DPO and there is no waveform record to read. **Load trace...** puts a captured or saved waveform there instead.

A verdict badge is drawn at the top right of the graticule. Where the instrument's own counter has a tally, that is the verdict; failing that, the drawn samples are tested against the shapes here. An empty mask is not a pass, and nothing is shown for one.

Setting Up an Eye

An eye mask is judged against a folded eye diagram, and the instrument has to be set up for one: the sweep positioned so the middle of a bit lands on the mask, DPO on where it is fitted, and the hit counter started. **Start measurement** does all of it. Two things cannot be discovered and are asked for first.

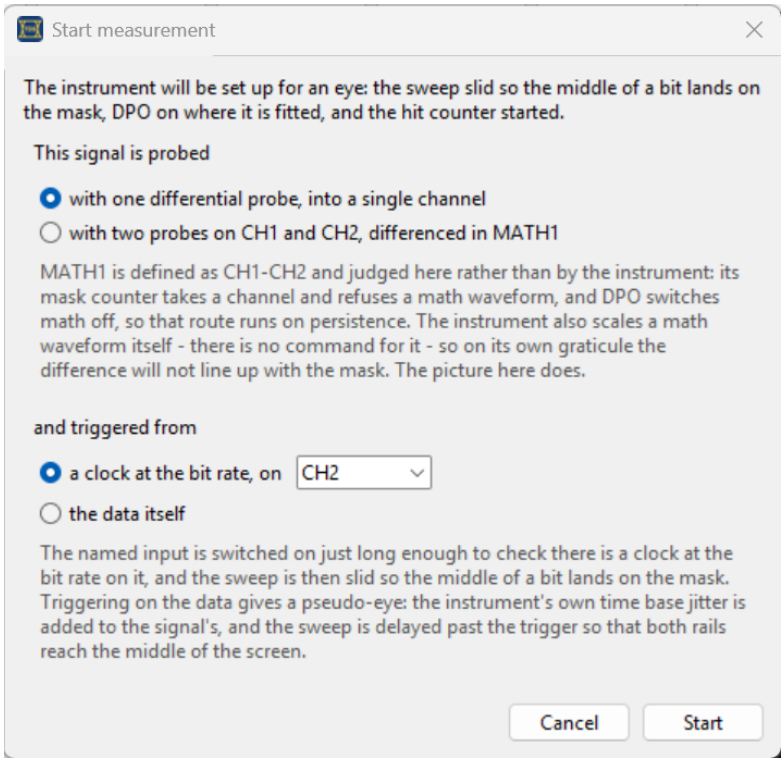


Figure 6–2: The Start measurement dialog for an eye mask

How it is probed Whether the signal arrives on one differential probe or as two probes to be differenced in MATH1. Asked only where the mask's setup says the mask is drawn for a differential pair.

What it is triggered from A clock at the bit rate on a named input, a clock to be looked for, or the data itself.

Neither can be found out by asking the instrument: it cannot see what is on the end of a probe, and an input with nothing plugged in reads the same as one whose clock is switched off.

With a Clock

The named input is switched on just long enough to check that there is a clock at the bit rate on it, and switched off again. A square wave at the bit rate crosses its own middle every half a bit, evenly; data crosses at whole multiples of a bit and unevenly, and the gaps tell the two apart.

Naming the input skips the search for a clock, but not the check. An input with nothing on it is reported rather than triggered on, because the instrument would sit waiting for an edge that is never coming.

The trigger level is measured rather than set with the instrument's own set-to-50% - `TRIGGER:MAIN SETLEVEL`, the front panel's Set 50% button - because measuring gives an exact level from the captured clock without the clock having to stay the live trigger source. The clock's own trace is then switched off.

Without a Clock

Triggering on the data can only ever show half an eye: the bit after a rising edge is always high. The instrument's own answer is the delayed time base, and the program uses it: trigger on a data edge and look sixteen bits past it, by which point the data no longer depends on the triggering edge, so ones and zeroes both land in the middle. The delay is then tuned against the mask itself.

Checking the Sweep

An eye diagram needs about one bit across the graticule. The program measures how many the sweep is holding and says so if it is far off, because a screen holding twelve bits, or a twelfth of one, will not produce an eye.

NOTE. *On the two-probe route the instrument's own counter is not used. MASK:SOURCE will not take MATH1, so the counter would be tallying one leg of the pair against a mask drawn for the difference. That route runs on persistence and is judged here instead.*

Mask Files

Two formats are read and written, both with the extension `.MSK`, and the format of each file is identified from its contents. Saving keeps the format the file came in as.

This program's format	Plain text: a TDSMASK header, the name, the source, and one line per segment holding its points. Keeps a shape exactly, including a concave one.
Tektronix i-Pattern	The binary format TTiP used, and the only Tektronix mask format for this generation. Its count field allows ten masks of 50 points, and the 2023-byte file every shipped mask uses has room for nine.



CAUTION. *An i-Pattern file keeps a mask's points without their order, so a concave shape may not come back as it was drawn. The program orders the points by angle about their middle, which recovers a convex shape exactly and a concave one only by luck. It warns before saving a concave shape in that format.*

A setup file may sit beside a mask, under the same name with the extension `.SET`, the way Tektronix shipped one with every mask. **Save setup...** reads the instrument's current settings, shows them, and writes one.

The dialog is a starting point rather than the answer. A mask is often drawn before the signal it is for exists, and the setup beside it must describe what the mask means rather than what the instrument happened to be showing, so every field can be changed before it is written. A field left empty is not written at all.

Unit interval, seconds	The bit period the mask is drawn for. This is what makes a mask an eye mask, and the eye setup cannot be done without it. No instrument setting records it, so it can only be typed here.
The signal is differential	Whether the mask is drawn for a signal across a pair. It decides what Start measurement asks for and how it sets the instrument up.

NOTE. *Those two are written as REM lines, which the instrument ignores and this program reads. A setup carrying them is still a setup the instrument will load.*

What the Program Sends

A mask is sent as one command per segment, for all eight segments, and the display is then turned on:

```
MASK:MASK1:POINTSPCNT <x,y,x,y,...>
MASK:MASK2:POINTSPCNT <x,y,x,y,...>
   ... through to MASK:MASK8
MASK:DISPLAY ON
```

Every segment is written, the empty ones as a single `0.0,0.0`, which is how the instrument says there is nothing there. Writing only the filled ones would leave whatever the last mask put in the others, and half of somebody else's mask is worse than none. **Clear** is the same eight commands with every segment empty.



CAUTION. `MASK:STANDARD` *is never sent. It deletes the existing mask and puts a standard one in its place, so sending it after the points throws them away. A user mask is points, with the standard left alone.*

Reading the instrument's segments back asks `MASK:STANDARD?` first, under a timeout of its own. An A series has no mask subsystem and does not answer it, and the answer is remembered so that the list does not spend most of a minute saying so every time it refreshes. Then one query per segment:

```
MASK:STANDARD?            (is there a mask subsystem at all)
MASK:MASK1:POINTSPCNT?
   ... through to MASK:MASK8
```

Judging a Signal

Start measurement puts the instrument where it can count, then zeroes its tally and starts it:

```
HORIZONTAL:MODE MAIN
LIMIT:STATE OFF
ACQUIRE:STOPAFTER RUNSTOP
ACQUIRE:STATE RUN
DISPLAY:MODE INSTAVU        (read back; see below)
MASK:COUNT:STATE 0
MASK:COUNT:STATE 1
*OPC?
```

The display mode is read back rather than assumed. Where DPO is fitted the instrument tests a few hundred thousand acquisitions a second instead of a few hundred; where it is not, the mode does not take and `DISPLAY:PERSISTENCE 5` is set instead, so the picture still accumulates.

NOTE. *There is no* `MASK:COUNT:RESET` *on this firmware - it answers "undefined header" - so the state going off and on again is the only way to clear the counters. Started here and read after the capture, the count covers exactly the time the picture was being built, which makes it the verdict for that picture.*

The verdict is read with two queries; the second is what makes a verdict possible at all:

```
MASK:COUNT:WAVEFORMS?      (how many acquisitions were tested)
MASK:COUNT:TOTAL?          (how many of them touched the mask)
```

A total of zero means nothing touched the mask only if something was tested, so the waveform count is read first and a count of zero acquisitions is reported as no verdict rather than as a pass.

Error Log

The instrument keeps its own error history in non-volatile memory. This tab reads it, saves it, and clears it. It is not the event queue this program watches while it works, and it is not a log of anything this program did.

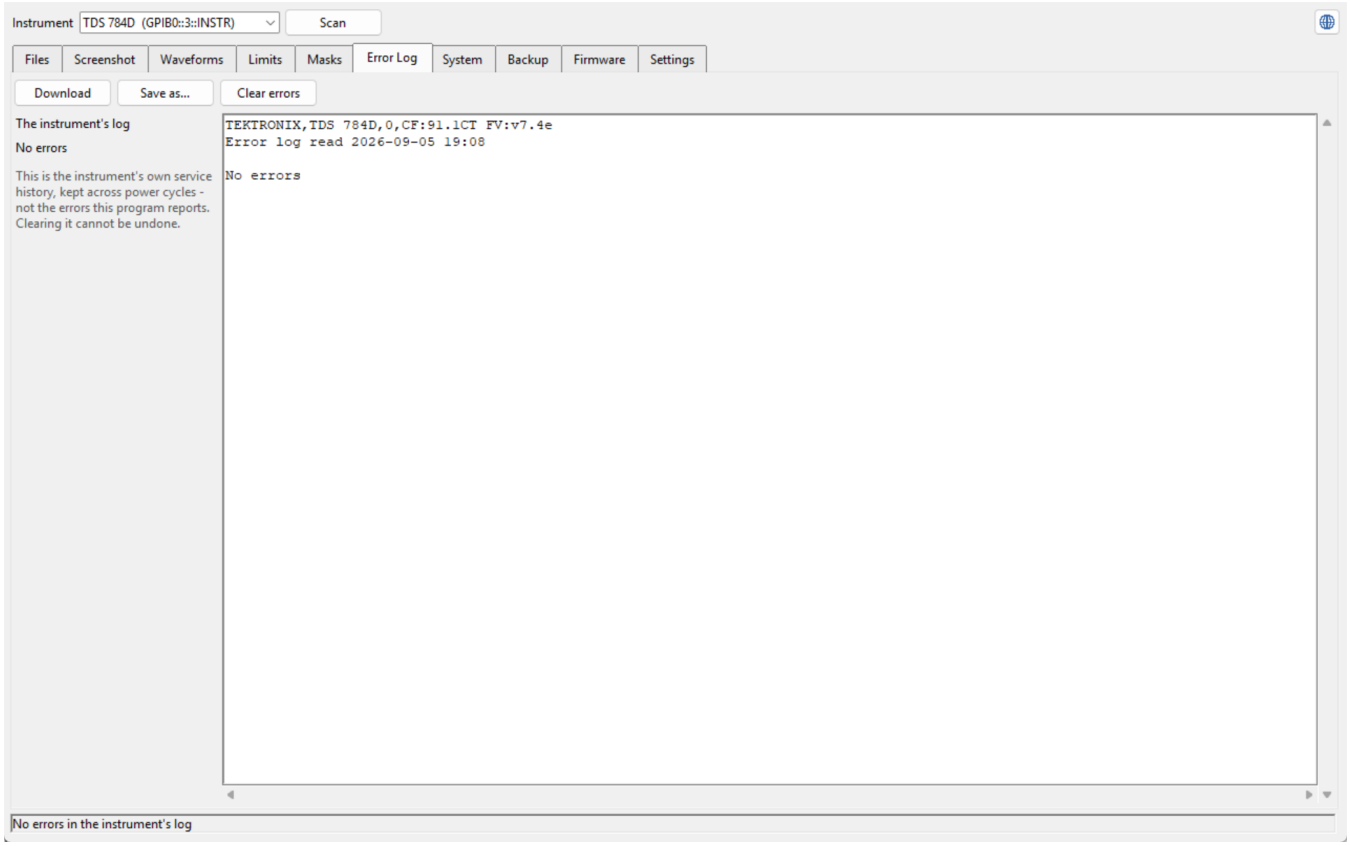


Figure 7–1: The Error Log tab

Reading the Log

Press **Download**. The whole log is read, oldest entry first, and shown under **The instrument's log** numbered from 1. The text is the instrument's own wording, unaltered: nothing is reworded, filtered or left out.

The pane opens with the instrument's identity and the time the log was read, and closes with "End of errors". A log with nothing in it reads "No errors".

The pane is read-only but can be selected and copied. Arrow keys, Home, End, Page Up, Page Down and Ctrl+C all work, and a right-click over the pane offers **Copy**, greyed unless something is selected.

NOTE. *Reading is not instant. There is no command that returns the whole log: it is read one entry at a time, and 31 entries from a TDS 640A took 3.24 seconds. Reading stops after 2000 entries, which is a guard against firmware that never says it has finished rather than a limit anybody should reach.*

Memory Faults

Where any of the entries record a failed memory test, a line above them says how many. The count is in the saved file as well as on screen.

Nothing is hidden or reordered to produce it. The count is made by looking through the entries for the instrument's own sub-test names, taken from the firmware rather than guessed at:

```
digAcqMemAddrDiag     dsyRastModeV0Walk    nvRamDiag
digAcqMemDataDiag     dsyRastModeV1Walk    nvLibrariansDiag
digAcqMemPatDiag      dsyDiagRasRegMem
digAtSpeedAcqMemDiag  dsyDiagPPRegMem
```

The first column is the acquisition memory, the second the video RAM, and the third the non-volatile RAM - `nvLibrariansDiag` is its own checksum test. The acquisition names carry a letter for the channel, `digAcqMemAddrDiagA` and so on, so they are matched anywhere in a line rather than whole.

NOTE. *Naming a memory test is not the same as failing one. A log that has just been initialised opens with "WARNING: 600 error log initialized, nvRamDiag error log re-initialized", which names one and is the instrument saying it has begun keeping records. An entry is counted only if it says ERROR as well.*

Saving the Log

Press **Save as...**. The file is plain text and is named after the instrument and stamped with the date and time, for example: `TDS784D_errorlog_20260821-210345.txt`.

Clearing the Log



WARNING. *Clearing the error log erases the instrument's error history. It is kept across power cycles and cannot be recovered. Download and save it first if there is any chance it will be needed.*

Press **Clear errors** and confirm. The program clears the log, reads it back and reports the result. If the instrument still reports entries afterwards, the program says so, with any reason the instrument gave.

Instruments With No Error Log

Where the firmware has no ERRLOG subsystem the pane reads "This firmware has no error log." and **Download** and **Clear errors** are greyed for the session.

What the Program Sends

There is no command that returns the whole log; it is read entry by entry:

```
ERRLOG:FIRST?      the oldest entry, and positions the cursor
ERRLOG:NEXT?       repeatedly, until an empty reply
```

Every entry arrives wrapped in double quotes, which are SCPI delimiters and are stripped. An empty log answers with an empty quoted string, not with silence; silence means the firmware has no ERRLOG at all.

Clearing is `ERRLOG CLEAR`, an argument to the header rather than a subcommand. `ERRLOG:CLEAR` does not exist and answers event 113, "Undefined header". The event queue is drained before and after.

System

The System tab holds the housekeeping the instrument keeps in non-volatile memory: its identity, the front-panel lock, its clock, the hardcopy and RS-232 ports, signal path compensation, the extended diagnostics, a secure erase, a factory recall, and a factory option unlocker.

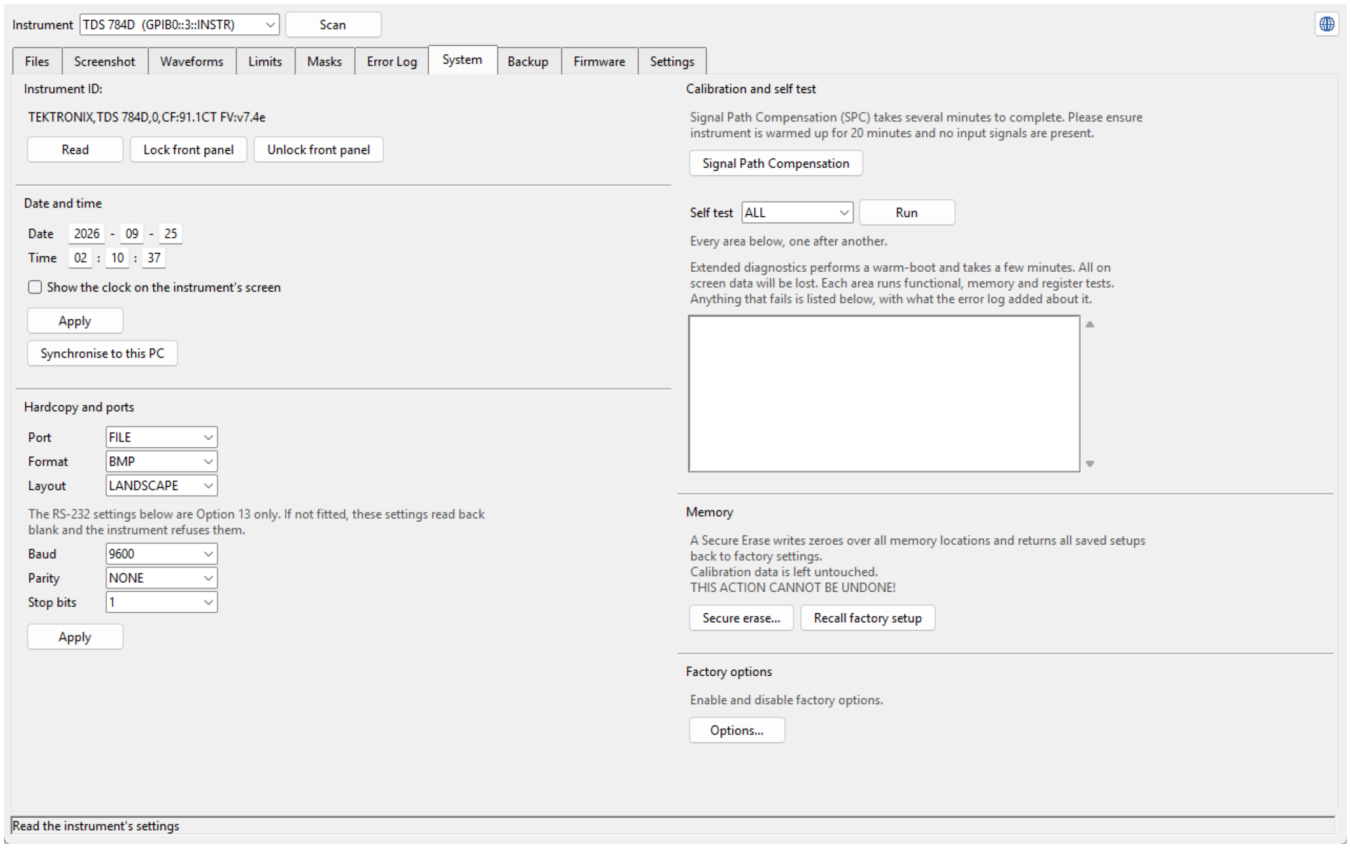


Figure 8–1: The System tab

NOTE. *The tab reads the instrument once, the first time it is brought to the front. A dozen queries at every start-up would waste time, and most sessions never open this tab. Press* ***Read*** *to read it again, for example after changing instruments.*

Identity

Instrument ID: shows the reply to `*IDN?` and, under it, the reply to `ID?`, which is where the instrument lists the options it believes it has.

Read Read every setting on this tab from the instrument.

Lock front panel Send LOCK ALL. The instrument's own controls stop responding until it is unlocked.

Unlock front panel Send UNLOCK ALL.

Date and Time

The instrument's clock is set in separate boxes rather than as one string, because the instrument accepts yyyy-mm-dd and nothing else. A single-digit month is padded to two digits before it is sent.

1. *Set the clock from this computer:* Press **Synchronise to this PC**. The date and time are taken at the moment of the press.
2. *Or set it by hand:* Type into the boxes and press **Apply**.

Show the clock on the instrument's screen is sent with either, and on its own by **Apply**. Empty date or time boxes are not sent: an instrument sent 0000-00-00 would accept it.

Hardcopy and Ports

The hardcopy port, format and layout are the instrument's own settings, the ones the Screenshot tab borrows and puts back. Settings made here also apply to the instrument's own front-panel **HARDCOPY** button.

Table 8–1: Hardcopy and RS-232 settings

Setting	Choices
Port	GPIB, RS232, CENTRONICS, FILE
Format	BMP, BMPCOLOR, TIFF, PCX, PCXCOLOR, EPSIMAGE, INTERLEAF, THINKJET, DESKJET, LASERJET, EPSON
Layout	LANDSCAPE, PORTRAIT
Baud	300, 600, 1200, 2400, 4800, 9600, 19200
Parity	NONE, EVEN, ODD
Stop bits	1, 2

NOTE. *The RS-232 settings are Option 13 only. Where the option is not fitted they read back blank, and a blank box is not sent. Any setting the instrument refuses is reported, and the refusals are collected and shown together rather than stopping at the first.*

Calibration and Self Test

Signal Path Compensation

Signal path compensation corrects the instrument's own signal path for its present operating temperature. Press **Signal Path Compensation** and confirm.



CAUTION. *Allow the instrument its twenty-minute warm-up and disconnect every input signal before compensating. Any signal present during compensation is built into the correction.*

The command is `*CAL?`, and the program allows it ten minutes. A reply of 0 is a pass; anything else means the instrument could not compensate itself, and the self test below will say more about why. Nothing else can use the bus while it runs.

Self Test

Choose an area and press **Run**. The areas are ALL, ACQUISITION, DISPLAY, CPU and FPANEL, and a line under the picker says what the selected one covers. The result and the instrument's own log of it appear in the pane below.

Each area runs functional, memory and register tests. The two that carry a memory array are offered first: ACQUISITION runs the acquisition memory's address, data, pattern and at-speed tests, one set per channel, and DISPLAY runs the video RAM's walking-bit, register and pattern tests over the raster. Testing memory alone is not offered, because the instrument does not offer it: ALL is the only argument its own selection commands accept.

Reading a Failure

The instrument's result log names modules and says pass or fail against each. What failed inside a module is written to the instrument's error log, which is why a failing module reads "(see error log)". The program reads that log before and after the run and lists whatever the run added underneath the module list, so both halves of the answer are in one place. Failing lines are shown in red.

The detail is worth reading in full. A display memory failure reads:

```
ERROR: diagnostic test failure, dsyRastModeV0Walk,
raster fail @ 1,1 read: 8, expect: f
```

That is the sub-test's own name, where in the raster it failed, what was read there and what should have been read. Reading 8 where f was expected means three bits are stuck low. Right-click the pane to copy a line.

The module named in the result log is a board, not a test. A raster failure like the one above reads **FAIL ++ Processor**, because on this generation the display and the processor are one large circuit board and a video RAM fault is a fault on it. The CRT driver is a board of its own and is not covered by any of this: the data sent to it goes one way only, so there is nothing to read back and nothing to test.



CAUTION. *An area stops at its first failure. The instrument's diagnostic executive carries on while a test returns 0x00, 0x44 or 0xA4, and jumps to the next area on anything else - 0xA3 is the usual failure. So the tests after the first one to fail never ran, and their absence from the list does not mean they passed. A short list of failures does not mean the instrument is nearly healthy.*

There is no way round this from the bus. The instrument's own selection commands take ALL and nothing else, so no single test can be run on its own, and a second fault in the same area shows itself only once the first is repaired and the area is run again. The pane says so under every failure.



CAUTION. *The extended diagnostics warm-boot the instrument. Whatever is on its screen is lost, and it shows its startup screen for two minutes or so while it restarts. Allow about three minutes for the whole operation.*

The program says nothing at all to the instrument for the first two and a half minutes, and asks only every fifteen seconds after that. This is deliberate. An instrument being questioned repeatedly while it restarts can stop responding altogether and need to be power cycled; a TDS 754D did exactly that, twice, when an earlier version asked it every two seconds. Left alone, the same instruments restart and answer normally.

NOTE. *An instrument that passes returns to a live trace on its own and needs no key press. An instrument that fails stops at the diagnostic log on its own screen and waits there for CLEAR MENU on the front panel. The bus is not affected either way: the flag and the log are read from an instrument waiting at that prompt exactly as from one back at its trace.*

If the instrument has still not answered when the program gives up, look at its screen rather than at the computer: from the bus, an instrument that is waiting at a prompt and one that has stopped responding look exactly the same. Press CLEAR MENU if a diagnostic log is showing; switch it off and on if it is frozen. Nothing is lost either way. Once it is back, press **Run** again to read the result: the flag, the log and the error log all still hold it.

NOTE. *A line in the result pane reading "pass -- Cal Initialization (see error log)" is a pass. The words in brackets are part of the test's name, not a report of a failure.*



CAUTION. *Diagnostics can reset the instrument's calibration librarians. Back up the calibration constants from the Backup tab before running them on an instrument whose calibration matters.*

Memory



WARNING. *A secure erase writes zeroes over every waveform and setup memory and returns all saved setups to factory settings. Calibration data is left untouched. This cannot be undone.*

Press **Secure erase...** and confirm. The command is TEKSECURE and the program allows it five minutes. The instrument reports the result as an event: 2285 is a pass, 2286 is a failure to verify the memory afterwards.

Recall factory setup sends FACTORY, which returns the instrument to its default settings. The GPIB address, the calibration constants and any protected user data are left untouched.

The calibration constants used to be read and written from this tab. They are on the Backup tab now, in Section 9, beside everything else that is copied off the instrument and put back.

Factory Options

The instrument's options are held as constants in non-volatile memory. This dialog writes them. Nothing here is in any Tektronix manual; the words, their values and their meanings were read out of the firmware images and confirmed on the bench.

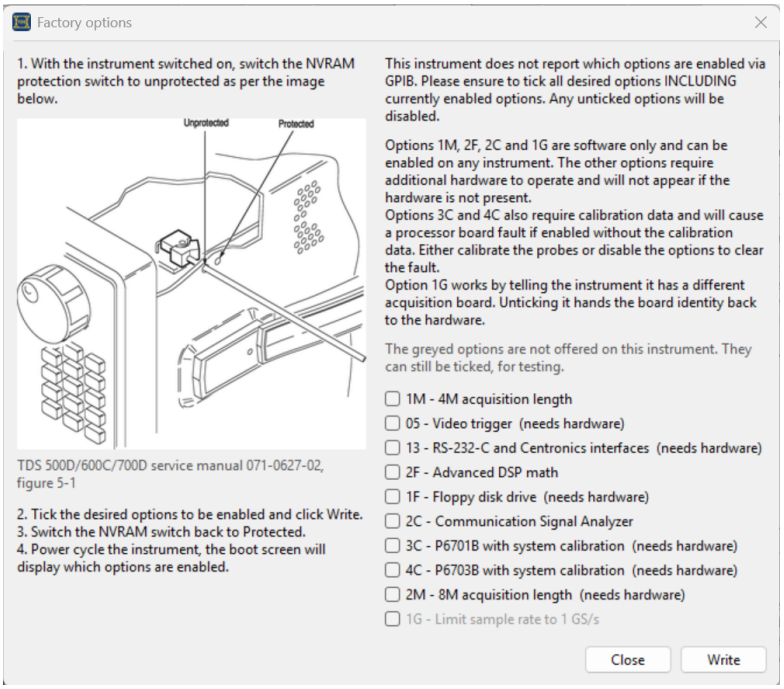


Figure 8–2: The Factory options dialog

NOTE. Options... *is greyed on the earliest firmware. v2.16e has no service password, so the words cannot be written there at all - the same fact that greys the calibration constants in Section 9.*

WARNING. *Enabling an option the instrument's hardware cannot support does not add the hardware. Options 3C and 4C raise a processor board fault if they are enabled without their calibration data; either calibrate the probes or disable the options again to clear the fault.*

The Procedure

1. *Unprotect the memory:* With the instrument switched on, set the NVRAM protection switch on the side of the cabinet to unprotected. See Figure 8–3.
2. *Choose the options:* Tick every option that is to be enabled, including the ones already enabled. Anything left unticked is disabled.
3. *Write them:* Press **Write** and confirm.
4. *Protect the memory again:* Set the switch back to protected.
5. *Restart the instrument:* Power cycle it. The boot screen lists the options that are enabled.

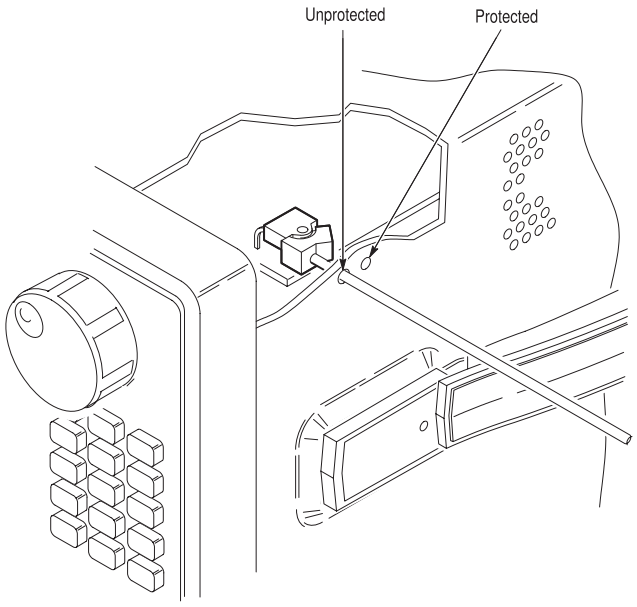


Figure 8–3: Accessing the protection switch, from Tektronix service manual 071-0627-02



CAUTION. *With the switch protected, the writes are accepted and nothing changes. Nothing can be read back to confirm what was written: the instrument does not report its option words, and* `ID?` *reports only what the firmware believes. The boot screen is the only report.*

The Option Words

Every word is written each time, ticked or not: the value below if ticked, and 0 if not.

Table 8–2: The factory option words

Code	Option	Word	On
1M	4M acquisition length	327686	1
05	Video trigger	327687	1
13	RS-232-C and Centronics interfaces	327688	1
2F	Advanced DSP math	327689	1
1F	Floppy disk drive	327690	1
2C	Communication Signal Analyzer	327692	1
3C	P6701B with system calibration	327693	1
4C	P6703B with system calibration	327694	1
2M	8M acquisition length	327695	1
1G	Limit sample rate to 1 GS/s	131219	14

Options 1M, 2F, 2C and 1G are software only and take effect on any instrument whose firmware reads them. The others need hardware that must be present.

NOTE. *1G is not an option word at all. The firmware reports 1G when the acquisition board identifies itself as board 14; word 131219 is the constant that overrides that identity when it is not zero. Writing 14 tells the instrument it has a different acquisition board; writing 0 hands the identity back to the hardware.*

An option greyed in the dialog is one this instrument is not offered. For every code but 1G that means the firmware has no getter for it and will never report it however its word is set; Appendix A lists what each firmware reads. 1G is offered only on the TDS 540B, where it was traced, and the 520D, 540D, 724D and 754D, which Tektronix lists it for. A greyed option can still be ticked and written, which is how you find out whether a particular instrument takes it.

What the Program Sends

Everything this tab does is one of the commands below. A read is one query; a setting written is followed by `*OPC?`, and any event the instrument raises in reply is collected and shown as a refusal rather than stopping the rest.

```
*IDN?                    identity
ID?                      the fitted-option string
DATE?   TIME?            the clock
DISPLAY:CLOCK?           clock shown on screen, or not
HARDCOPY:PORT?   :FORMAT?   :LAYOUT?
RS232:BAUD?  :PARITY?  :STOPBITS?  :HARDFLAGGING?  :SOFTFLAGGING?
```

Reading the Tab

The RS-232 queries read back blank where the option is not fitted, and a blank is not written back.

```
LOCK ALL                 lock the front panel
UNLOCK ALL               release it

DATE "2026-09-05"
TIME "19:03:08"
DISPLAY:CLOCK ON         or OFF
HARDCOPY:PORT  :FORMAT  :LAYOUT    the value set beside each
RS232:BAUD  :PARITY  :STOPBITS     the value set beside each

*CAL?                    signal path compensation; 0 is a pass
DIAG:SELECT:<area> ALL   choose the area
DIAG:STATE EXECUTE       run it - this warm-boots
DIAG:RESULT:FLAG?        pass or fail, once it is back
DIAG:RESULT:LOG?         the module log

TEKSECURE                secure erase; event 2285 is a pass
FACTORY                  recall the factory setup

PASSWORD PITBULL
*OPC?
WORDCONSTANT:ATPUT 327686, 1      (and so on, all ten)
*OPC?                             (after each)
```

Factory Options

The password unlocks the constant-writing commands and is sent before the first word.

Backup

The Backup tab copies the instrument's contents and settings into a file on the computer, and writes them back.

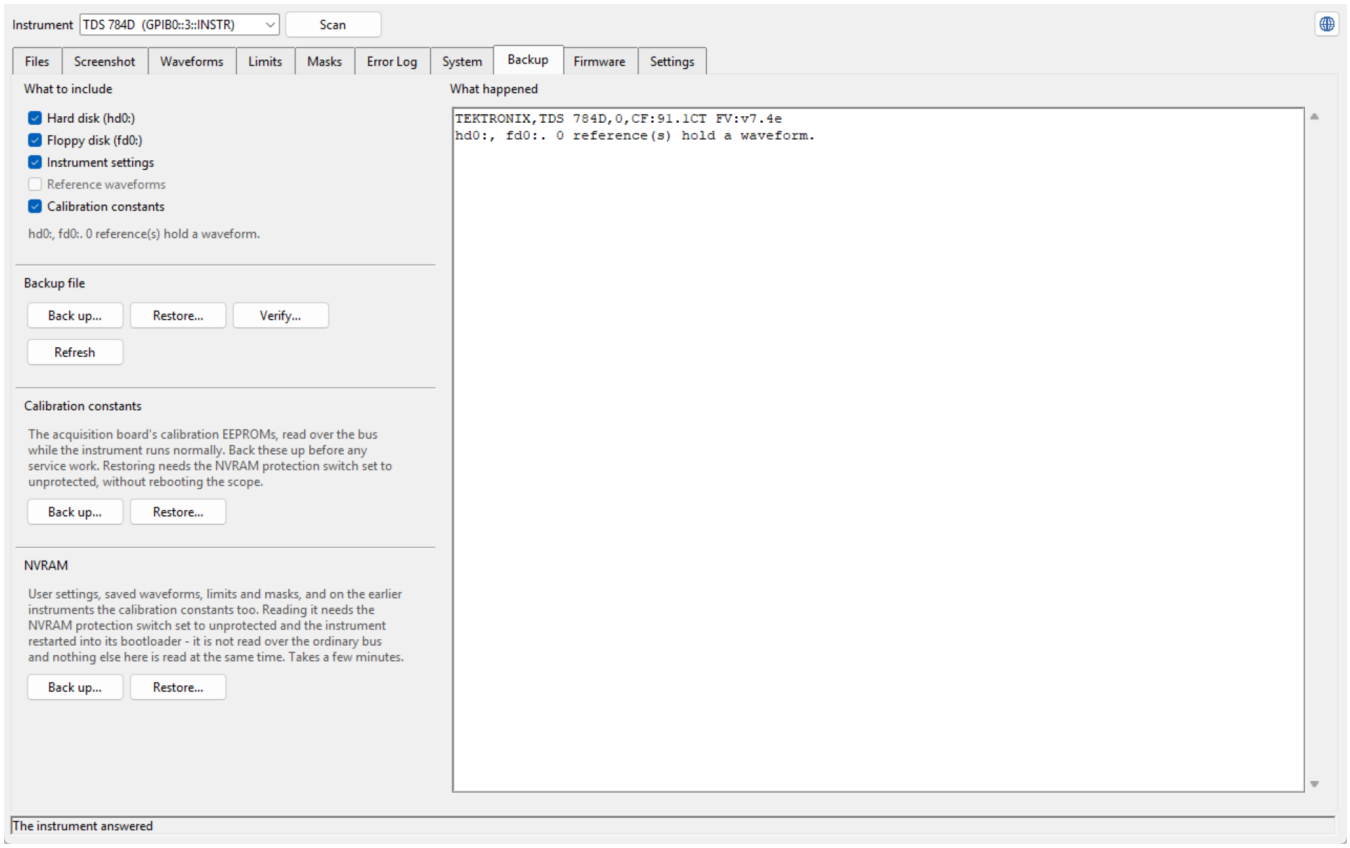


Figure 9–1: The Backup tab

Tick what to include and press **Back up...**, which asks where the file goes. The report on the right says what came across and what did not.

Everything on this tab writes into that report, and it is kept rather than replaced: each step is named as it starts, the result follows it, and a job that fails writes the reason there as its last line. A blank line separates one run from the next. The status line below shows the same steps with their running counts, but only the current step.

NOTE. *Press* ***Refresh*** *first. The two volume boxes - hard disk and floppy - start unticked; Refresh asks the instrument, ticks whichever volumes it finds and greys the ones it does not. The references box is greyed the same way when no stored reference holds a waveform. Settings and calibration are always readable and stay ticked.*

What Goes In

Hard disk (hd0:)	Every file on the instrument's disk, in the folders it keeps them in.
Floppy disk (fd0:)	The same for a floppy, if there is one in the drive.
Instrument settings	The reply to `*LRN?`: every setting the instrument will report.
Reference waveforms	Each stored reference that holds a waveform, as a .wfm file.
Calibration constants	The acquisition board's 252 words, read twice and compared.

NOTE. *The error log is not included. The Error Log tab, in Section 7, reads, saves and clears it.*

The Backup File

The file is an ordinary zip. Inside, the disk is under `hd0/`, a floppy under `fd0/`, the settings in `setup.set`, the references in `refs/` and the calibration words in `cal.bin`.

Beside them is `manifest.json`: what the instrument was - its identity, its options, its clock, its hardcopy and RS-232 settings, and when the backup was taken - and a checksum for every file in the bundle.

NOTE. *The hardcopy and RS-232 settings are recorded because* `*LRN?` *does not carry them, nor the date and time. Without the manifest they would not be written down anywhere.*

NOTE. *A zip's own checksum catches a damaged archive. It says nothing about a file that was read wrongly from the instrument, and a bad read over GPIB is silent. The manifest makes that detectable, and* ***Verify...*** *performs the check. It also records which instrument a bundle came from and what was fitted to it, so that years later a folder of identically named zips is not a folder of guesses.*

There is no disk image, because this firmware has no command that reads a sector. Files are the whole of what can be copied.

Verifying

Press **Verify...** and choose a backup. Every file in it is read and compared with the checksum recorded when it was made, and the report says how many were checked and names any that did not match. No instrument is needed.

Restoring



WARNING. *Restoring writes to the instrument. Files of the same name are overwritten, and there is no undo.*

1. *Choose what to restore:* Tick it on the left. Anything the backup does not hold is skipped.
2. *Choose the backup:* Press **Restore...**. The file is verified before anything is written.
3. *Confirm:* The confirmation names the parts that will be written and the file they come from.

Files go back through the same verified write the Files tab uses: each is written, read back and compared. Folders are made as they are needed.

The settings are sent as separate commands rather than as the one long message the instrument answered with. A command this firmware does not support is refused and named; sent as one message, a single refusal takes the rest of the message with it and nothing says which command it was. This is what makes it practical to apply one instrument's setup to another.

NOTE. *The calibration constants are never part of a restore. They have their own button, below, for the reasons under Calibration Constants.*

Calibration Constants

The acquisition board keeps its calibration constants in two small EEPROMs; boards mark them U1052 and U1055, or U1055 and U1056. The firmware reaches them through a librarian it exposes as a service command, so they are read and written over the bus with the instrument running normally. Nothing in this section requires the ROM monitor or the cabinet to be opened.

NOTE. *Both buttons are greyed on the earliest firmware. v2.16e has no constant librarian and no service password, so there is nothing there to read or write; the factory options in Section 8 are greyed on the same instruments for the same reason. Appendix A lists what each firmware answers.*



WARNING. *The accuracy of every measurement the instrument makes depends on these constants. Back them up before any service work, and restore only a backup taken from the same instrument.*

Backing Up

1. *Read them:* Press **Back up...** in the Calibration constants box. The 252 words are read, read a second time and compared.
2. *Save them:* Choose a file. A name made from the model and the date is offered.

The file is 512 bytes: eight bytes of padding, then the words in the order the librarian holds them, high byte first. This is the same file the reference tool writes, and answers to the same checksum verifier. Ticking **Calibration constants** above puts the same 512 bytes into a backup as `cal.bin`.

NOTE. *If every word reads back the same value, this is reported. Calibration data is not one number repeated, so that is a sign the librarian address is wrong for this generation. The address is documented for the TDS 500B, 600A and 700A families and is an assumption elsewhere.*

The Checksum

Word 0 is the 16-bit sum of the 251 words after it, across both EEPROMs rather than one each. A block that disagrees with it is damaged, and the program says so when a backup is saved and asks about it before a restore is written.



CAUTION. *A file that repeatedly fails its checksum is worth keeping as evidence of what an instrument held. It is not worth restoring to a working one: writing it leaves the acquisition board holding the same damage.*

Restoring

Press **Restore...**, choose a backup and confirm. No copy is taken first. The only undo is whatever **Back up...** beside it wrote, and the confirmation says so before anything is written.

All 252 words are written, not only the ones that differ. Rewriting a cell refreshes the charge holding it, so a wholesale write leaves the whole block freshly written instead of leaving the untouched words to go on ageing.

Written Is Not Stored

A word sent with `WORDCONSTANT` goes into a working copy in the instrument's memory. It does not reach the EEPROMs on the acquisition board until `CALIBRATE:STORE?` is sent, and that command answers 0 when the constants have been committed and something else when they have not. The program sends it at the end of every restore and says which answer it got.



WARNING. *Reading a word back proves it arrived in the working copy and nothing more, because the copy is what a read returns. A restore can write all 252 words, read every one of them back correctly, and agree with its own checksum, while the instrument stores none of it.*

NOTE. *Tektronix's own field adjustment software ends every adjustment with the same command and treats any non-zero answer as a fault.*



CAUTION. *Set the NVRAM protection switch to unprotected before restoring, as described under Factory Options in Section 8, and set it back afterwards. With the switch set to protected, the words are accepted but nothing is stored.*

The NVRAM

The non-volatile RAM holds what the instrument remembers when it is switched off: front-panel settings, saved setups, stored waveforms, limit templates, masks, the service log, enabled options, and on the earlier instruments the calibration constants as well. It is battery-backed static RAM, and a byte written to it is a byte changed - there is no erase and no undo.



WARNING. *This box talks to the ROM monitor, not to the running firmware. The instrument has to be started into its bootloader and answers at GPIB address 29 while it is there, so nothing else on this tab works at the same time. Section 10 describes how to start it in the monitor and how to return it to normal operation.*

Backing Up

Press **Back up...** in the NVRAM box and choose a file. The 640 kB that hold the two memory parts are read in full, read a second time, and the two compared. Any byte outside the clock that changed between the two reads is reported as a bus fault and the backup is refused rather than saved.

The monitor answers for a whole megabyte, but no instrument in the range has more than 640 kB fitted, and what sits above that is either empty or the larger part answering a second time. Both are reproduced by a restore of the 640 kB below, so the backup reads the top only far enough to confirm it is one or the other and leaves it out of the file. Should it ever find something up there that is neither, it reads the whole megabyte instead - the part nobody can re-derive is never the part left out.

NOTE. *The first fourteen bytes are the clock. They tick while the read is going on, which is expected and reported as such, not as a fault.*

The comparison knows where the clock is rather than tolerating a few moved bytes anywhere, so a genuine bus fault is still caught while the ticking clock is not.

What the Checksums Say

When a file is chosen for restore, its checksums are verified before it is offered for writing. Each firmware keeps its NVRAM in named sections - the hardware accountant, the internal and external constants, the state, the environment, the diagnostics - and each section carries a sixteen-bit sum of itself in the two bytes before it. The tables come out of the firmware images themselves, fifteen distinct ones across the thirty-one images in the archive.

The report names the firmware the file matches and how many of its sections add up. A file that matches nothing known is not refused - but it is not written without a second confirmation that says so.

NOTE. *A section of nothing but zeros adds up to zero, which agrees with a check word of zero under every table. Those are counted separately and reported, because a wiped NVRAM would otherwise read as one whose every checksum is valid.*

Restoring

1. *Choose a file:* Press **Restore...** in the NVRAM box. The file's checksums are verified and the report appears on the right.
2. *Confirm:* Twice if the checksums do not match a known firmware, once if they do.
3. *The window is measured:* See below.
4. *It is written, and read back:* Any difference outside the clock stops the restore and says where.



CAUTION. *No copy is taken first. A restore is written over what is there, and the undo is whatever* ***Back up...*** *in the NVRAM box wrote earlier. Take one before restoring.*



CAUTION. *Restore only a backup taken from the same instrument. The checksum report can tell a damaged file from a sound one; it cannot tell whose instrument a sound one came from, and the sections sit at different addresses across the generations.*

Why Less Is Written Than the File Holds

The monitor answers for a megabyte, and no instrument in the range has a megabyte fitted. A TDS 680B has a 128 kB part at the bottom of that window and a 512 kB part above it - 640 kB in all - and the top 384 kB of the window is the larger part answering a second time. A TDS 640A has 512 kB and repeats from the base.

So writing a whole dump back would write those cells twice. On the instrument the dump came from the two copies agree and no harm is done. On one laid out differently the low part of the window is written correctly and then overwritten by the top of the same file, and only the read-back afterwards would say so.

The program therefore measures how much of the window is distinct memory before it writes, by writing four bytes at one address and reading them at another. It cannot be read out of the data: blank memory reads alike at every address whether it is aliased or not, and blank is what an instrument needing a restore often is. The measurement is taken after the backup, so the four bytes it disturbs are already saved, and it puts them back either way.

A restore then stops at that boundary. Restoring a one-megabyte backup to a TDS 680B writes 640 kB, and the tab says so under the result. Nothing is missing: the rest of the file is the same memory over again.

NOTE. *It is also the earliest test of whether writing works at all. With the protection switch protected the four bytes read back unchanged, and the restore stops there - a second or two in, rather than after a minute of writing that could never have succeeded.*

A file whose repeated region differs from this instrument's came from an instrument with another memory layout, and is refused before anything is written.

What the Program Sends

```
PASSWORD PITBULL
WORDCONSTANT:ATOFFSET? 262144,0        (252 of them)
WORDCONSTANT:ATOFFSET 262144,0,4660    (all 252)
WORDCONSTANT:ATOFFSET? 262144,0        (read back, each one)
CALIBRATE:STORE?                       (commit; 0 is stored)
```

Calibration Constants

None of this is in a Tektronix manual, but the last line is in Tektronix's own software: the Field Adjustment Software 063-3289-01 ends every adjustment with `CALIBRATE:STORE?` and stops if the answer is not 0. The query and the file layout are those of the getcaldata program in `ragges/tektools`. The setter is the set form of the same query, taking the same base and offset, which was established on a TDS 640A by writing a word, reading it back changed, and writing it back again.

NOTE. *The password is sent only if the first word is refused without it. A TDS 640A on v3.8.8e refuses; an instrument that answers has nothing to unlock and is not sent a command it may not know.*

The NVRAM

The monitor's own packets, the same ones the firmware writer in Section 10 uses. A read asks for 1024 bytes at a time and a write carries 512, which is not the same number by accident: 1024 is what the monitor sends, and a write has to be received into that same buffer behind twelve bytes of header. A 1024-byte write payload overruns it. Measured on a TDS 680B, where it left the monitor waiting for the rest of a packet it had been promised, and only a power cycle recovered it.

NOTE. *The clock is never written. Its fourteen bytes are the instrument's own time and its command and watchdog registers, not somebody's data, and writing a backup's copy of them back would set the instrument to whenever the backup was taken.*

Firmware

The Firmware tab writes a firmware image to the instrument. It does not do so over GPIB in the ordinary way, because firmware cannot replace itself. It writes through the ROM monitor the instrument starts in when it is switched on with its NVRAM protection switch set to unprotected.

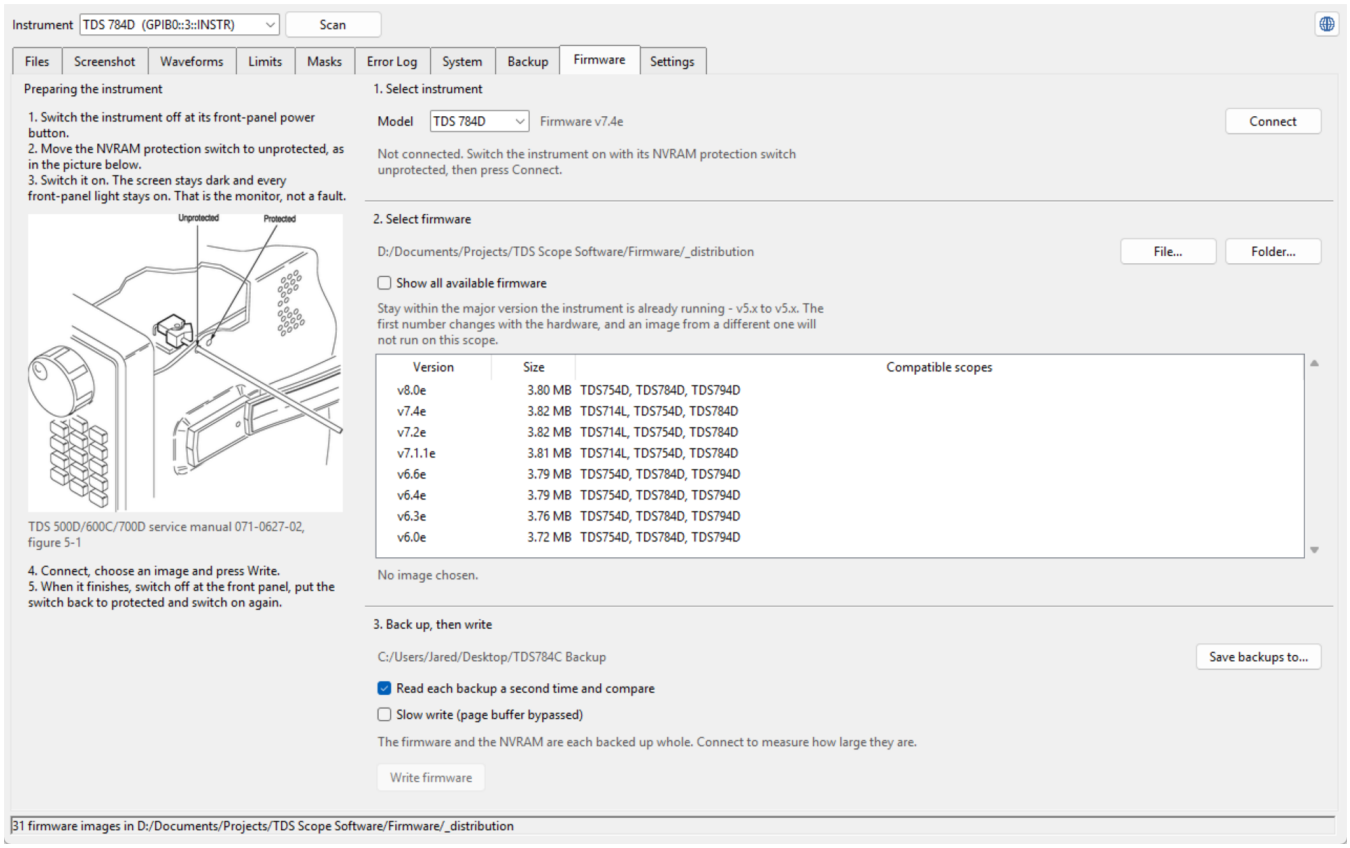


Figure 10–1: The Firmware tab



CAUTION. *The write erases the instrument's firmware flash completely before writing anything. Do not interrupt it at the instrument. An interrupted write leaves an instrument that will not start; see Recovery at the end of this section.*

Preparing the Instrument

The Protection Switch

The NVRAM protection switch is a rocker reached through two small access holes in the right side of the cabinet, near the front. It is the same switch the factory options in Section 8 require. The tab shows the service manual's illustration of it.

Setting the switch to unprotected and applying power starts the monitor instead of the firmware. In that state the instrument appears faulty: the screen stays dark, every front-panel indicator stays lit, and the instrument does not answer at its usual GPIB address. It answers at address 29.

Starting the Monitor

1. *Switch the instrument off:* The front-panel power button is sufficient.
2. *Set the switch to unprotected:* As illustrated on the tab.
3. *Switch the instrument on:* The screen stays dark and the front-panel indicators stay lit. This is the monitor, not a fault.
4. *Connect:* Press **Connect**. The tab reports the flash type fitted, how much of it is fitted, and the first four bytes of the firmware already in it.

NOTE. *An instrument still running its firmware is reported as such, with the address at which it answered. The two states are distinguishable: an instrument running firmware answers* `*IDN?` *at its ordinary address, and one in its monitor answers at address 29 and nowhere else.*

Controls

The right-hand side of the tab has three numbered boxes, used in order: **1. Select instrument**, **2. Select firmware** and **3. Back up, then write**. The left-hand column is the preparation described above and does not change.

Model — The instrument the image list is filtered to. Taken from `*IDN?` where the program has connected to the instrument in the ordinary way during this session, and manually entered where it has not. The monitor identifies nothing.

Firmware version — Beside the Model box: the version the instrument reported at `*IDN?` before its protection switch was moved. It reads *firmware not read* where the program has not connected to the instrument in the ordinary way this session.

Connect — Find the monitor, identify the flash, and measure how much of it is fitted.

Folder...	Select the folder or archive holding the firmware images.
File...	Select one image directly, from anywhere on the computer. It is listed first and remains listed whichever model is selected.
Show all available firmware	List every image in the folder rather than only those shipped for the selected model.
Save backups to...	Select the folder the backups are written to. Required.
Read each backup a second time and compare	Verify each backup before anything is erased. Leave ticked.
Slow write (page buffer bypassed)	Drive each page from the computer instead of from the instrument. See *Fast and Slow Writes*.
Write firmware	Perform the complete operation. Available only when the monitor has answered, an image is selected, and a backup folder is set.

Firmware Images

Where They Are Kept

A collection of images is published with this program as a single archive. Press **Folder...** and select the folder it is in. The folder may hold loose `.bin` files, an archive of them, or both. An archive is read without being unpacked, and only the image being written is decompressed.

An image held anywhere else, such as one downloaded on its own or one on removable media, is selected with **File...** instead. It is added to the top of the list and remains listed whichever model is selected, because it was chosen by name rather than found. A file that does not begin like a firmware image is reported before it can be used.



CAUTION. *Stay within the major version the instrument is already running: v5.x to v5.x, and so on. The first number changes with the hardware generation, and an image from one will not run on an instrument from another. The version the instrument reported is shown beside the Model box for exactly this comparison.*

One Image, Several Models

Tektronix shipped one binary for a whole instrument family, so the same image is found under several model names. Which instruments an image was shipped for is therefore taken from an index file supplied with the images, not from the filename. The list shows one row for each distinct image under **Version** and **Size**, with every model it was shipped as under **Compatible scopes** - the selected model included, so the row answers "will this run on the instrument in front of me" on its own.

NOTE. *The version is read from the filename, or from the string inside the image where the name carries none. Where the two disagree, the tab shows both and picks neither.*

Backups

Two backups are written before anything is erased: the instrument's NVRAM to one file, and its firmware flash to another. Both cover the whole of the part concerned. Neither is optional, and how much is copied is not a setting: erase is chip-wide, so a partial backup of the flash would not restore the instrument.

How much flash is fitted is measured at **Connect** rather than assumed. An address beyond the end of the array reads back as one inside it, so the array is as long as the first two-megabyte boundary whose contents repeat the base.

NOTE. *The backups are not deleted afterwards. Keep the firmware backup: it is the image that was in the instrument, and writing it back is how the instrument is returned to what it was.*

Writing

The Sequence

One press performs the complete operation, in the order below. It stops at the first step that does not complete correctly.

1. *Back up:* NVRAM and firmware are read into the backup folder.
2. *Verify the backups:* Each is read a second time and compared. A disagreement stops the operation here, before anything is erased.
3. *Erase:* The flash is erased. A part reporting an erase failure stops the operation.
4. *Write:* The image is written a page at a time. A page the image leaves blank is not written; the erase has already left the part in that state.
5. *Verify:* The flash is read back and compared against the image, blank pages included.

When the operation completes, switch the instrument off at the front panel, return the protection switch to protected, and switch it on again.

Fast and Slow Writes

Driving the flash's page buffer from a computer costs about seventeen bus turnarounds for every 512 bytes, nearly all of them four-byte writes. The program avoids this by placing a page loop in the instrument's own scratch memory and calling it once per page, which reduces a page to one turnaround.

Slow write (page buffer bypassed) drives each page from the computer instead. It is roughly five times slower. It is worth setting on an instrument this program has not written before, because it leaves one fewer untried component.

Table 10–1: Complete operation, measured

Instrument	Flash fitted	Image	By instrument	By computer
TDS 784D	28F160S5	3.82 MB	276 s	not offered
TDS 784C	28F016SA	3.36 MB	255 s	652 s
TDS 640A	28F008SA × 4	2.34 MB	243 s	not offered

NOTE. *The setting is available only where the flash has a page buffer. A part without one can be driven from a computer only a longword at a time, which would take days, so on those instruments the setting is unavailable and the tab gives the reason. Of the three instruments measured, only the TDS 784C has a page buffer.*

Recovery

The monitor is held in mask ROM and does not depend on the flash, so an instrument whose firmware is partly written is not lost. Switch it off at the front panel, leave the protection switch unprotected, switch it on again, and write an image: either the one that failed, or the backup taken at the start of the run.

Where the final verification reports a difference, the tab gives the address of the first byte that differs and the same instruction. Nothing is lost; write it again.

Settings

The Settings tab holds what belongs to this program rather than to the instrument: the colours everything is drawn in, the size of a saved picture, and where the settings and the log are kept.

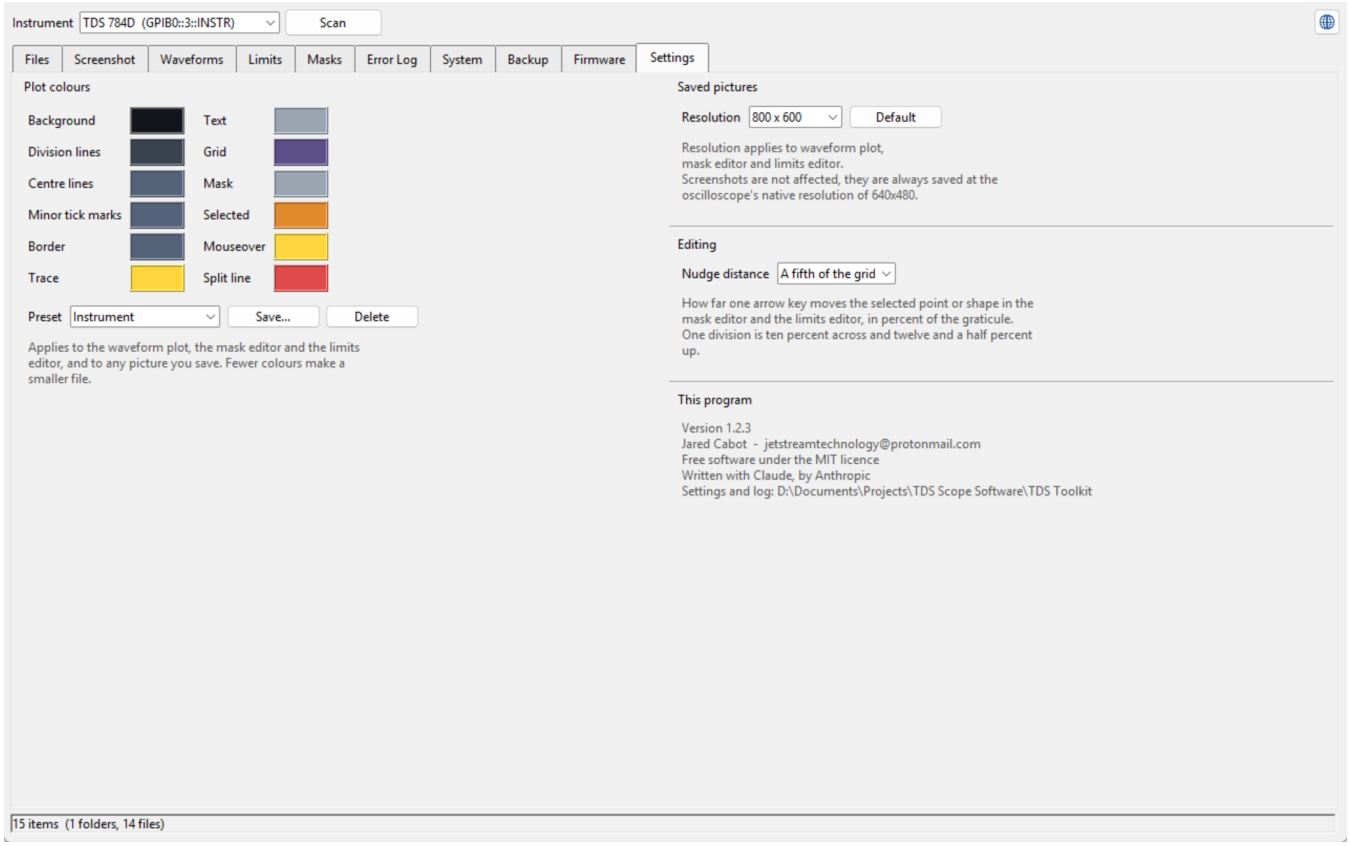


Figure 11–1: The Settings tab

Plot Colours

Twelve colours are used by the waveform plot, the mask editor, the limits editor and every picture saved from them. Click a swatch to change it. There is nothing to apply and nothing to cancel: a colour takes effect as it is picked.

Table 11–1: What each colour draws

Colour	Where it is used
Background	Behind the graticule
Division lines	The graticule's own lines
Centre lines	The heavier cross through the centre
Minor tick marks	The small ticks along the centre cross
Border	The graticule's outer border
Trace	A captured trace, where the scheme is not the instrument's own
Text	Captions, markers and readouts
Grid	The editor's drawing grid, and the template band
Mask	Mask and limit shapes
Selected	A point or shape that is selected
Mouseover	Whatever the pointer is over
Split line	The line the scissors would cut along

Presets

Four schemes are supplied:

Instrument The instrument's own screen colours, dark. Under this scheme a captured trace is drawn in the colour the instrument itself draws that channel in.

Printed page Dark on white, for printing and for pasting into a document.

Phosphor Green on near-black, like an old phosphor CRT.

High contrast White on black, with no half tones.

Save... keeps the colours you have chosen under a name of your own, and **Delete** removes one you have saved. The four supplied schemes cannot be deleted, but saving one of the same name replaces it.

Decode colours

The protocol-decode chevrons take their colour from the decoder itself, not from a setting here: each protocol carries its own colour so several decoded buses on one screen can be told apart, and a frame with a bad checksum, parity or the like is drawn in red. There is nothing to set for them on this tab.

Saved Pictures

The **Resolution** box sets the size of a picture saved from the waveform plot, the mask editor or the limits editor. The sizes offered are 640 x 480, 800 x 600, 1024 x 768, 1280 x 960, 1600 x 1200 and 1920 x 1440; the default is 800 x 600. All are in the instrument's native 4:3 aspect ratio.

NOTE. *This does not apply to screenshots. A screenshot is the instrument's own picture and is always saved at its native resolution of 640 x 480.*

Editing

Nudge distance sets how far one arrow key moves the selected point or shape in the mask editor and the limits editor. It is a percentage of the graticule and is the same in both directions, so remember that a division is ten percent across and twelve and a half percent up.

The first entry, **A fifth of the grid**, is the default and follows whatever the editor's grid spacing is, so five presses cross one square and the keys and the snapping agree with one another. Choosing a distance instead fixes it to a fraction of the grid.

Language

The globe button at the right-hand end of the bar above the tabs, marked **A**/文, lists every language file found beside the program, with a dot against the one in use. The choice is remembered. With none remembered the program follows the language Windows is set to, and falls back to English. Where there are no language files at all the menu says **No language files found** and the program stays in English.

Nine catalogues are supplied. A language can be added or corrected without rebuilding anything: a `lang` folder beside the program overrides the one inside it. Copy `en.json`, name it for the language tag, fill in the `_meta` block and translate the values, leaving the keys alone.

NOTE. *The keys are the English text, so anything left out falls back to English on its own and a half-finished file is useful straight away. Placeholders such as %s and %d must survive translation in the same number and, where they are not named, in the same order. Run the program with --check-translations to audit every file before using it.*

Where Things Are Kept

The bottom box names the folder the program keeps its files in, which is the folder the program itself is in.

tdstoolkit.json	The settings. Written as they are changed. A file that cannot be read is renamed tdstoolkit.json.bad, and the program continues with the defaults.
tdstoolkit.log	What the program did, what the instrument reported that was not expected, and any internal fault. Moved to tdstoolkit.log.old at 512 kbytes; one generation is kept.
masks	The mask library.
decoders	The protocol-decode plugins, topped up from the bundled set each run.
capabilities.json	What is known about each instrument's firmware. A file here overrides the one inside the program, so an instrument can be added without rebuilding anything.

NOTE. *Settings are not saved if the folder cannot be written to. A read-only folder prevents saving but does not interrupt the session, so the program carries on without reporting it.*

Firmware Capability

The capabilities of an instrument are decided by its firmware. The table below was read out of the firmware images themselves: the SCPI keyword table decoded from the ROM says directly whether a command exists, and the option words say which option codes the firmware reads.

The program consults this table when it connects. An instrument that is not in it is asked instead, and works without any user intervention; an entry that turns out to be wrong is corrected against the instrument and written to the log. The table is `capabilities.json`, and a copy beside the program overrides the one inside it.

Files and Transfers

Read is the command a file's contents can be fetched with: READFILE where the firmware has it, PRINT where it does not, and none at all where neither exists. **Write** is whether FILESYSTEM:WRITEFILE exists.

Table A–1: Filesystem and transfer commands by firmware

Model	Firmware	Filesystem	Read	Write
TDS 520	v2.16e	no	none	no
TDS 520A	v3.8.7e	yes	PRINT	no
TDS 520B	v4.1e	yes	PRINT	no
TDS 520C	v5.0e	yes	READFILE	yes
TDS 520C	v5.1.1e	yes	READFILE	yes
TDS 520C	v5.2e	yes	READFILE	yes
TDS 520C	v5.3e	yes	READFILE	yes
TDS 524A	v3.8.2e	yes	PRINT	no
TDS 524A	v3.8.7e	yes	PRINT	no
TDS 540	v2.16e	no	none	no
TDS 540A	v3.8.3e	yes	PRINT	no
TDS 540B	v4.1e	yes	PRINT	no
TDS 540C	v5.0e	yes	READFILE	yes
TDS 540C	v5.1.1e	yes	READFILE	yes
TDS 540C	v5.2e	yes	READFILE	yes
TDS 540C	v5.3e	yes	READFILE	yes
TDS 544A	v3.8.4e	yes	PRINT	no
TDS 580C	v5.0e	yes	READFILE	yes
TDS 580C	v5.1.1e	yes	READFILE	yes
TDS 580C	v5.2e	yes	READFILE	yes
TDS 580C	v5.3e	yes	READFILE	yes
TDS 620	v2.04e	no	none	no
TDS 620A	v3.8.5e	yes	PRINT	no

Table A–2: Filesystem and transfer commands by firmware (cont.)

Model	Firmware	Filesystem	Read	Write
TDS 620B	v4.2e	yes	PRINT	no
TDS 620B	v4.4.1e	yes	PRINT	no
TDS 640A	v3.8.8e	yes	PRINT	no
TDS 644A	v3.8.4e	yes	PRINT	no
TDS 644A	v3.8.5e	yes	PRINT	no
TDS 644B	v4.2e	yes	PRINT	no
TDS 680B	v4.2e	yes	PRINT	no
TDS 680B	v4.4.1e	yes	PRINT	no
TDS 684B	v4.4.1e	yes	PRINT	no
TDS 684C	v5.1e	yes	READFILE	yes
TDS 694C	v6.4e	yes	READFILE	yes
TDS 714L	v7.1.1e	yes	READFILE	yes
TDS 714L	v7.2e	yes	READFILE	yes
TDS 714L	v7.4e	yes	READFILE	yes
TDS 744A	v1.0.1e	yes	PRINT	no
TDS 744A	v1.0e	yes	PRINT	no
TDS 744A	v1.1e	yes	PRINT	no
TDS 744A	v4.0e	yes	PRINT	no
TDS 744A	v4.2.1e	yes	PRINT	no
TDS 754C	v5.0e	yes	READFILE	yes
TDS 754C	v5.2e	yes	READFILE	yes
TDS 754C	v5.3e	yes	READFILE	yes
TDS 754D	v6.0e	yes	READFILE	yes

Table A–3: Filesystem and transfer commands by firmware (cont.)

Model	Firmware	Filesystem	Read	Write
TDS 754D	v6.3e	yes	READFILE	yes
TDS 754D	v6.4e	yes	READFILE	yes
TDS 754D	v6.6e	yes	READFILE	yes
TDS 754D	v7.1.1e	yes	READFILE	yes
TDS 754D	v7.2e	yes	READFILE	yes
TDS 754D	v7.4e	yes	READFILE	yes
TDS 754D	v8.0e	yes	READFILE	yes
TDS 784C	v5.0e	yes	READFILE	yes
TDS 784C	v5.2e	yes	READFILE	yes
TDS 784C	v5.3e	yes	READFILE	yes
TDS 784D	v6.0e	yes	READFILE	yes
TDS 784D	v6.3e	yes	READFILE	yes
TDS 784D	v6.4e	yes	READFILE	yes
TDS 784D	v6.6e	yes	READFILE	yes
TDS 784D	v7.1.1e	yes	READFILE	yes
TDS 784D	v7.2e	yes	READFILE	yes
TDS 784D	v7.4e	yes	READFILE	yes
TDS 784D	v8.0e	yes	READFILE	yes
TDS 794D	v6.0e	yes	READFILE	yes
TDS 794D	v6.3e	yes	READFILE	yes
TDS 794D	v6.4e	yes	READFILE	yes
TDS 794D	v6.6e	yes	READFILE	yes
TDS 794D	v8.0e	yes	READFILE	yes

Factory Options

An X means that firmware reads that option's word. An option with no X has no getter in the ROM, so the instrument will never report it however its word is set. This says nothing about whether the hardware the option needs is fitted.

Table A–4: Option codes each firmware reads

Model	Firmware	1M	05	13	2F	1F	2C	3C	4C	2M	1G
TDS 520	v2.16e	X	X	X	X						
TDS 520A	v3.8.7e	X	X	X	X	X					
TDS 520B	v4.1e	X	X	X	X	X					X
TDS 520C	v5.0e	X	X	X	X	X	X	X	X		X
TDS 520C	v5.1.1e	X	X	X	X	X	X	X	X		X
TDS 520C	v5.2e	X	X	X	X	X	X	X	X		X
TDS 520C	v5.3e	X	X	X	X	X	X	X	X		X
TDS 524A	v3.8.2e	X	X	X	X	X					
TDS 524A	v3.8.7e	X	X	X	X	X					
TDS 540	v2.16e	X	X	X	X						
TDS 540A	v3.8.3e	X	X	X	X	X					
TDS 540B	v4.1e	X	X	X	X	X					X
TDS 540C	v5.0e	X	X	X	X	X	X	X	X		X
TDS 540C	v5.1.1e	X	X	X	X	X	X	X	X		X
TDS 540C	v5.2e	X	X	X	X	X	X	X	X		X
TDS 540C	v5.3e	X	X	X	X	X	X	X	X		X
TDS 544A	v3.8.4e	X	X	X	X	X					
TDS 580C	v5.0e	X	X	X	X	X	X	X	X		X
TDS 580C	v5.1.1e	X	X	X	X	X	X	X	X		X
TDS 580C	v5.2e	X	X	X	X	X	X	X	X		X
TDS 580C	v5.3e	X	X	X	X	X	X	X	X		X
TDS 620	v2.04e	X	X	X	X						

Table A–5: Option codes each firmware reads (cont.)

Model	Firmware	1M	05	13	2F	1F	2C	3C	4C	2M	1G
TDS 620A	v3.8.5e	X	X	X	X	X					
TDS 620B	v4.2e	X	X	X	X	X					
TDS 620B	v4.4.1e	X	X	X	X	X					
TDS 640A	v3.8.8e	X	X	X	X	X					
TDS 644A	v3.8.4e	X	X	X	X	X					
TDS 644A	v3.8.5e	X	X	X	X	X					
TDS 644B	v4.2e	X	X	X	X	X					
TDS 680B	v4.2e	X	X	X	X	X					
TDS 680B	v4.4.1e	X	X	X	X	X					
TDS 684B	v4.4.1e	X	X	X	X	X					
TDS 684C	v5.1e	X	X	X	X	X					
TDS 694C	v6.4e	X	X	X	X	X	X	X	X		X
TDS 714L	v7.1.1e	X	X	X	X	X	X	X	X	X	X
TDS 714L	v7.2e	X	X	X	X	X	X	X	X	X	X
TDS 714L	v7.4e	X	X	X	X	X	X	X	X	X	X
TDS 744A	v1.0.1e	X	X	X	X	X					
TDS 744A	v1.0e	X	X	X	X	X					X
TDS 744A	v1.1e	X	X	X	X	X					X
TDS 744A	v4.0e	X	X	X	X	X					X
TDS 744A	v4.2.1e	X	X	X	X	X					X
TDS 754C	v5.0e	X	X	X	X	X	X	X	X		X
TDS 754C	v5.2e	X	X	X	X	X	X	X	X		X
TDS 754C	v5.3e	X	X	X	X	X	X	X	X		X
TDS 754D	v6.0e	X	X	X	X	X	X	X	X		X

Table A–6: Option codes each firmware reads (cont.)

Model	Firmware	1M	05	13	2F	1F	2C	3C	4C	2M	1G
TDS 754D	v6.3e	X	X	X	X	X	X	X	X	X	X
TDS 754D	v6.4e	X	X	X	X	X	X	X	X	X	X
TDS 754D	v6.6e	X	X	X	X	X	X	X	X	X	X
TDS 754D	v7.1.1e	X	X	X	X	X	X	X	X	X	X
TDS 754D	v7.2e	X	X	X	X	X	X	X	X	X	X
TDS 754D	v7.4e	X	X	X	X	X	X	X	X	X	X
TDS 754D	v8.0e	X	X	X	X	X	X	X	X	X	X
TDS 784C	v5.0e	X	X	X	X	X	X	X	X		X
TDS 784C	v5.2e	X	X	X	X	X	X	X	X		X
TDS 784C	v5.3e	X	X	X	X	X	X	X	X		X
TDS 784D	v6.0e	X	X	X	X	X	X	X	X		X
TDS 784D	v6.3e	X	X	X	X	X	X	X	X	X	X
TDS 784D	v6.4e	X	X	X	X	X	X	X	X	X	X
TDS 784D	v6.6e	X	X	X	X	X	X	X	X	X	X
TDS 784D	v7.1.1e	X	X	X	X	X	X	X	X	X	X
TDS 784D	v7.2e	X	X	X	X	X	X	X	X	X	X
TDS 784D	v7.4e	X	X	X	X	X	X	X	X	X	X
TDS 784D	v8.0e	X	X	X	X	X	X	X	X	X	X
TDS 794D	v6.0e	X	X	X	X	X	X	X	X		X
TDS 794D	v6.3e	X	X	X	X	X	X	X	X	X	X
TDS 794D	v6.4e	X	X	X	X	X	X	X	X	X	X
TDS 794D	v6.6e	X	X	X	X	X	X	X	X	X	X

Three points stand out. Option 2M is read only by v6.3e and later, so the 754D, 784D and 794D at v6.0e do not have it. Option 1G is on the 520B and 540B and most 744A versions, but not the 620B, 644B, 680B or 684B. And the 684C at v5.1e is the only C or D image with none of 2C, 3C, 4C or 1G.

File Formats

Waveforms

The formats this program reads and writes, and what each one keeps.

Table B–1: Waveform formats

Extension	Read	Write	What it holds
.isf	yes	yes	The instrument's own transfer format: the WFMPRE preamble and the curve exactly as CURVE? returned it. Byte for byte on a round trip
.csv	yes	yes	A units header, then scaled x,y pairs. Re-quantised when read back, to within one step of the eight-bit range
.tdw	yes	yes	The instrument's own layout under this program's own extension, so a file it wrote is never mistaken for one an instrument wrote. Sixteen-bit samples
.wfm	yes	no	A file an instrument wrote, which it can recall into a reference. Written only onto the instrument's own disk, by Send to instrument
.png	no	yes	The graticule as it stands, with the record strip, at the resolution set on the Settings tab

NOTE. *A waveform file from a TDS5000, TDS6000 or TDS7000, or from a DPO or MSO, is a different and much later format. The program recognises one and says so rather than misreading it.*

What a .WFM Carries

The instrument's own file holds more than the samples. All of the following is read out of it and written back into it:

Table B–2: The fields in a .WFM

Field	What it is
Record	The samples, sixteen bits each, with the sixteen acquired points either side of the record that the instrument uses to interpolate the display
Timebase	Seconds per sample, and the duration of the whole record
Trigger	Its position, as a whole percent of the record
Vertical	Volts per division, the offset in divisions, and the voltage at that offset
Units	Horizontal and vertical, separately
Source	Which input it came from: a channel, a maths waveform or a reference
Coupling	DC, AC or GND
Acquisition	Sample, Average, Envelope, Hi Res or Peak Detect
Format	Y or ENV

There is no XZERO and no date. The file holds the record and how to scale it, not when it was taken.

Masks and Setups

Table B–3: Mask and setup formats

Extension	What it holds
.MSK	A mask, in either this program's text format or the Tektronix i-Pattern binary format. The format is identified from the file's contents, and saving keeps the format it came in as
.SET	The instrument settings a mask was drawn against, as SCPI. Sits beside the mask under the same name. Two REM lines are written and read back by this program: the unit interval, and whether the signal is differential

Table B–4: Mask limits

Quantity	Value
Segments per mask	8
Points on segment	50
Minimum points on segment	2
Coordinates	0 to 100 percent, X across, Y up
Masks in an i-Pattern file	10 by the count field; 9 fit the 2023-byte file

Screenshots

A screenshot is saved either in the format the instrument sent or as PNG, and either way it carries any rotation or inversion applied here. Untouched, the native file is the instrument's own bytes exactly. The suffix offered for it is taken from those bytes, not from the format that was asked for.

The Program's Own Files

Table B–5: Files beside the program

File	What it is
tdstoolkit.json	Settings: language, address, colours, saved colour schemes, picture resolution, nudge distance, and the folders last used for firmware images, firmware backups and calibration backups
tdstoolkit.log	What was done and what the instrument reported. Renamed to .old at 512 kbytes
capabilities.json	The firmware capability table of Appendix A
masks	The mask library
lang	The language catalogues

If Something Goes Wrong

Connecting

Table C–1: The instrument does not answer

What you see	What to check
The picker opens by itself at start-up	Nothing answered at the remembered address. Press **Scan** in the picker; the instrument may have moved or be switched off
The scan finds nothing at all	The GPIB driver and the cable. VISA lists addresses it has been configured with whether or not anything is there, so an empty result means nothing on the bus answered
The scan finds other instruments but no oscilloscope	That the oscilloscope is in Talk/Listen, not Off Bus and not Hardcopy (Talk Only). In Talk Only it will not answer a query
An address answers as in use by another program	Another program holding the VISA session. Only one program at a time can have it

Transfers

Table C–2: Files and transfers

What you see	What it means
Upload is greyed for the whole session	This firmware has no FILESYSTEM:WRITEFILE. Every A and B series instrument is in this position. Nothing can send it a file over GPIB
Download is greyed for the whole session	This firmware has neither READFILE nor PRINT
"Nothing to browse" when connecting	Firmware v2.x, which has no filesystem commands at all. The Waveforms tab still works
"No disk in the drive"	The floppy drive was listed and answered, and holds no disk
An upload reports that it was not verified	The file was written and read back and the two did not match. Nothing is assumed: try again, and check the instrument's own screen

Waveforms and Screens

Table C–3: Capture

What you see	What it means
"Waveform capture is not supported in DPO mode"	In DPO the instrument fills a pixel database instead of a waveform record, and every point reads 128. Take a screenshot, or leave DPO
A source cannot be read	A channel that is not displayed cannot be read; the instrument answers event 2241. Choosing a greyed name switches it on
"This instrument offers no image format this program can read"	Every hardcopy format this firmware offers is a printer language rather than an image
A capture times out or the next command does	A hardcopy started too soon after the previous one is ignored, and a half-sent one blocks the bus. The program issues a device clear after a failure; give it a moment before trying again

Masks and Limits

Table C–4: Testing

What you see	What it means
The instrument draws the mask differently	It stores the points exactly and joins them its own way. Three arrangements are redrawn; the program names them before sending. Cutting the shape into simpler pieces avoids it
Send and Start measurement greyed for the whole session	This firmware has no Option 2C, so it has no mask subsystem to send to. Drawing, saving and loading still work; use the Limits tab to judge a signal
An eye that will not centre	Check that the sweep is holding about one bit. Send the setup beside the mask, which names the timebase the mask was drawn against
A limit test that passes everything	Check that the template was learnt from the same channel at the same volts per division. A template built against another channel's scaling lands in the wrong place

The Log

Anything unexpected goes to `tdstoolkit.log` beside the program, with the time and what was being done. An event the instrument raised that the program did not expect is shown once, written to the log, and put on the status line every time it happens afterwards. A session in which nothing goes wrong writes nothing at all.

A fault is written with a few lines in front of it saying what the program was connected to at the time: the instrument, the transfer commands its firmware has, the options fitted and the sources it was displaying. They are the first things anyone reading the fault needs, and by then it is too late to ask.

An internal fault in the program is caught, written to the log with its traceback, and reported without closing the window. Nothing is sent to the instrument in that case.